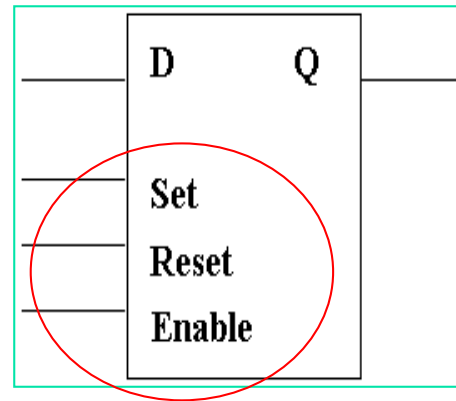


D-type Latch

- 範例2：具有「重置」、「設定」以及「輸出致能」訊號控制的準位感測門鎖之邏輯符號



// Latch_RS.V：具有「重置」、「設定」以及「輸出致／
//能」訊號控制的準位感測門鎖

```
module Latch_RS (Enable, Set, Reset, D, Q);
```

```
input  Enable, Set, Reset, D;
```

```
output Q;
```

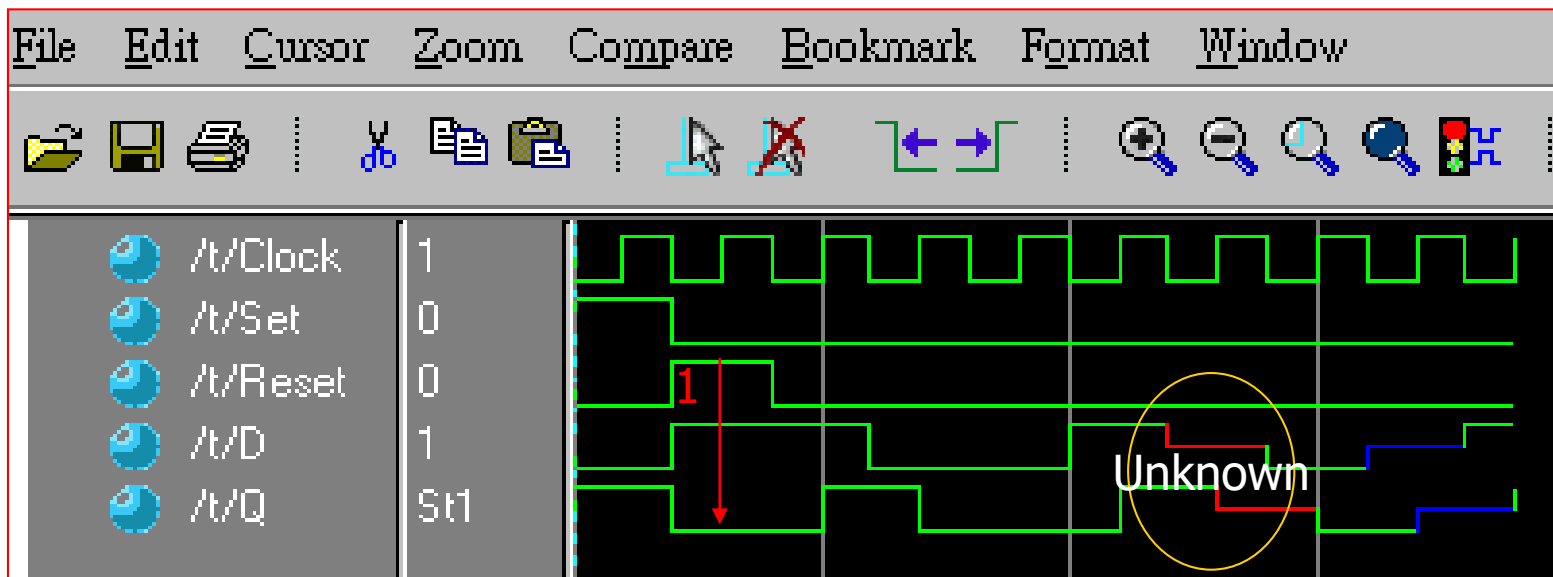
$$\left\{ \begin{array}{l} \text{Reset} = 1 \Rightarrow Q = 0 \\ \text{Reset} = 0 \Rightarrow \left\{ \begin{array}{l} \text{Set} = 1 \Rightarrow Q = 1 \\ \text{Set} = 0 \Rightarrow \left\{ \begin{array}{l} \text{Enable} = 1 \Rightarrow Q = D \\ \text{Enable} = 0 \Rightarrow Q = Q \end{array} \right. \end{array} \right. \end{array} \right.$$

```
    assign Q = Reset ? 0 : (Set ? 1 : (Enable ? D : Q));  
endmodule
```

Priority: Reset > Set > Enable

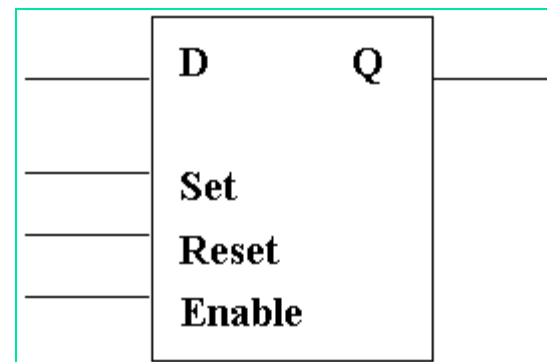
Simulation Waveform

4_02_LATCH_RS



```
module Latch_RS (Enable, Set, Reset, D, Q);  
input  Enable, Set, Reset, D;  
output Q;  
  
    assign Q = Reset ? 0 : (Set ? 1 : (Enable ? D : Q));  
endmodule
```

Clock=Enable

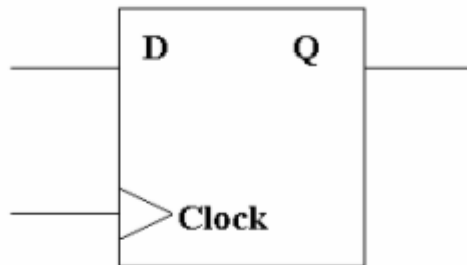


Enable 輸入埠不見了，自行模擬。

D-type Flip-flop (1)

- ❑ **正反器**：資料只對希望在某些控制訊號啟動時才進入閃鎖電路的雙穩態電路稱為正反器。
- ❑ **正反器與閃鎖 (Latch) 的區別**：正反器 (Flip-Flop) 是使用邊緣觸發 (Edge Trigger) 的方式動作的；而閃鎖 (Latch) 是使用準位觸發 (Level Trigger) 的方式動作的。
- ❑ **D 型正反器**為二進位儲存單元，其具有一個輸入端 (D)，時脈 (Clock Pulse) 訊號將輸入端的資料取樣後，儲存於正反器中，並且將之呈現在輸出端。
- ❑ **特性方程式**： $Q_{(t+1)} = D$ (其中：t+1 代表下一個時脈來臨的時間)

D 型正反器的邏輯符號



特性表

D	$Q_{(t+1)}$
0	0
1	1

D	Q(t)	Q(t+1)
0	0	0
0	1	0
1	0	1
1	1	1

D-type Flip-flop (2)

範例2：具有「重置」及「設定」控制訊號的正反器

//DFF_RS.V: 具有「重置」及「設定」訊號控制的正反器

```
module DFF_RS (Clock, Set, Reset, D, Q, Q_Bar);
```

```
input  Clock, Set, Reset, D;
```

```
output Q, Q_Bar;
```

```
reg    Q, Q_Bar;
```

```
always @(posedge Clock)
```

```
begin
```

```
    if (Reset) // 重置
```

```
    begin
```

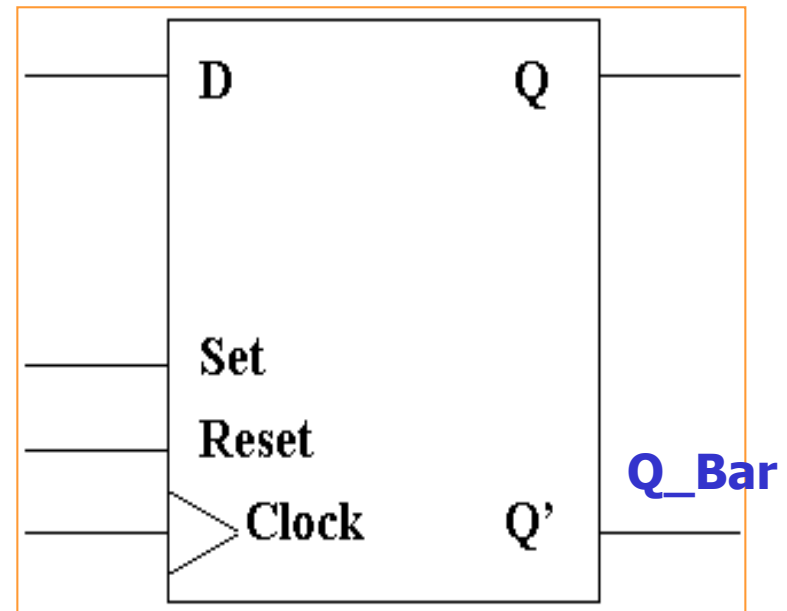
```
        Q = 0;
```

```
        Q_Bar = 1;
```

```
    end
```

```
    else
```

重要功能



D-type Flip-flop (3)

if (Set) // 設定

begin

重要功能

Q = 1;

Q_Bar = 0;

end

else

begin // 正常輸入 “D 至 Q”;

//Q_Bar = “D 取 Not”

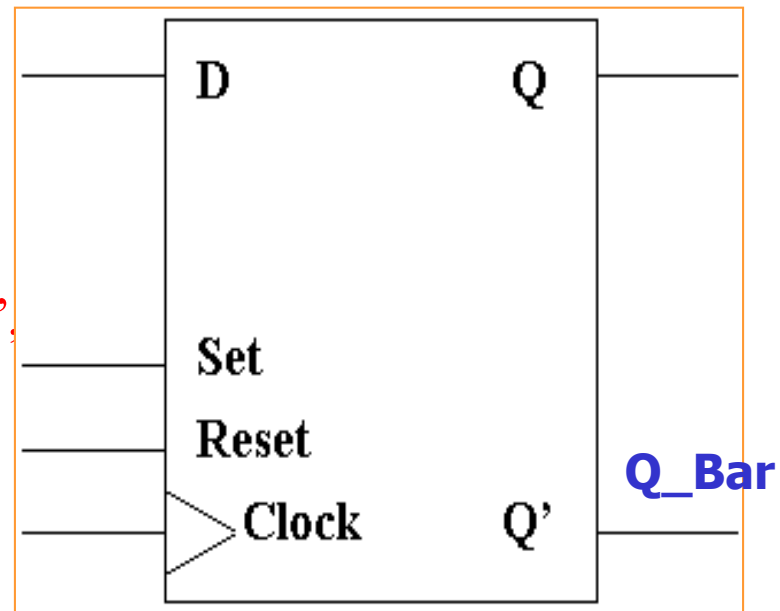
Q = D;

Q_Bar = ~D;

end

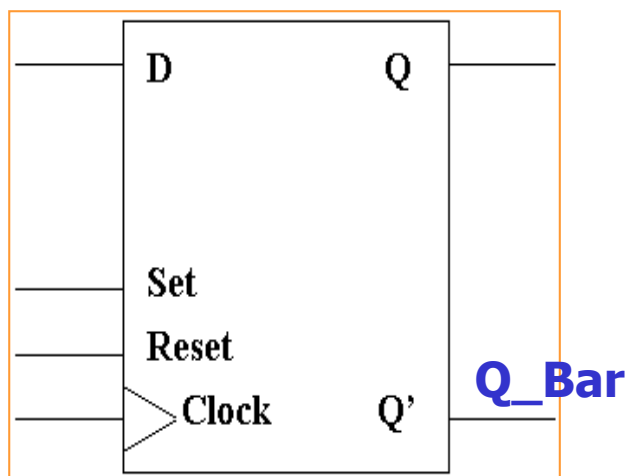
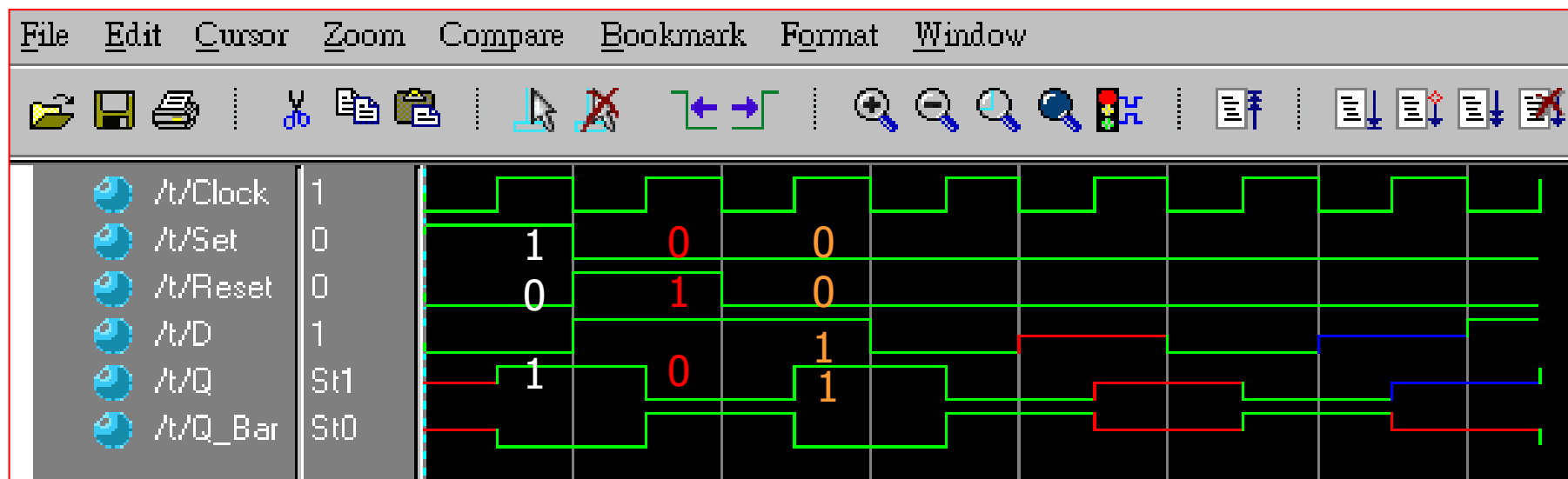
end

endmodule



Simulation Waveform

4_04_DFF_RS



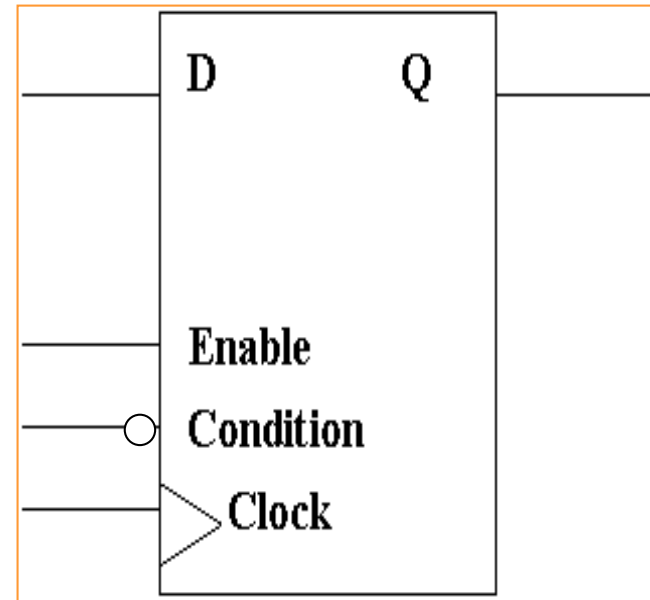
重點提示：

D_FF 具有除 2 功能，如何修正 Testbench 以顯示該項功能？

D-type Flip-flop (4)

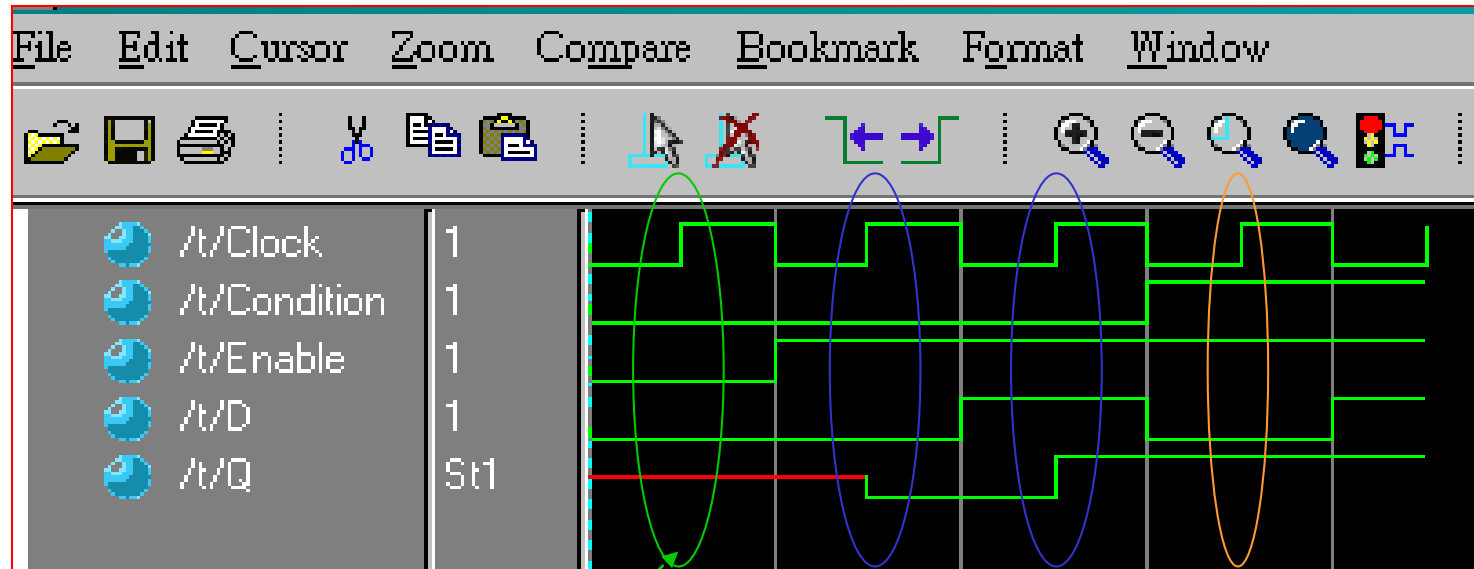
例3：具有「致能」輸出控制訊號的正反器

```
module Enable_FF1 (Clock, Condition, Enable, D, Q);  
input  Clock, Condition, Enable, D;  
output Q;  
reg    Q;  
  
always @(posedge Clock)  
begin  
    if (Enable) // 「致能」輸出  
        Q = (~Condition) ? D : Q;  
        // 若 Condition = 0, 則 Q = D  
    else  
        Q = 1'bx;  
        // 設定為x (不確定值, Unknown Value)  
    end  
end  
endmodule
```



Simulation Waveform

4_05_EN_DFF



```
always @(posedge Clock)
begin
    if (Enable) // 「致能」輸出
        Q = (~Condition) ? D : Q;
        // 若 Condition = 0, 則 Q = D
    else
        Q = 1'bx;
        // 設定為x (不確定值, Unknown Value)
end
```

Q = D

Q = Q



重點提示：

D_FF具有除2功能，如何修正Testbench以顯示該項功能？

D-type Flip-flop (5)

範例5: 具有**非同步(Asynchronous)重置(Reset)與預設(Preset)**的 D_FF

```
module D_FF_PR (Clock, Reset, Preset, D, Q);
```

```
input  Clock, Reset, Preset, D;
```

```
output Q;
```

```
reg    Q;
```

非同步用法

```
always @(negedge Reset or negedge Preset or posedge Clock)
```

```
begin
```

```
    if (!Reset) // Reset = 0
```

```
        Q = 1'b0; // 清除為 0
```

```
    else
```

```
        if (!Preset) // Preset = 0
```

```
            Q = 1'b1; // 設定為 1
```

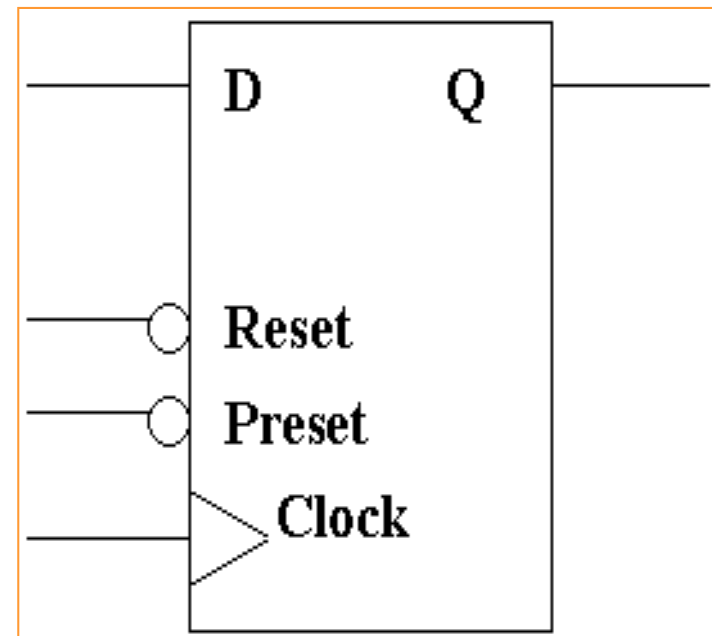
```
        else
```

```
            if (Clock) // Reset = 1
```

```
                Q = D; // 將 D 傳入 Q
```

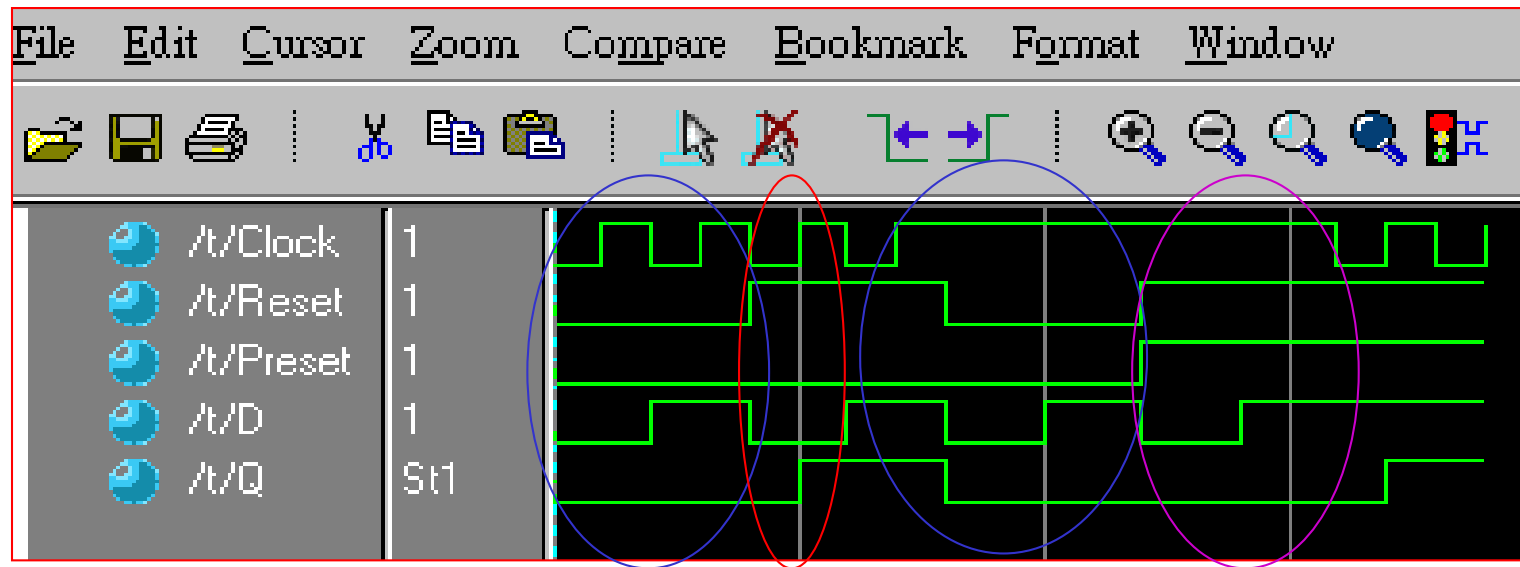
```
    end
```

```
endmodule
```



Simulation Waveform

4_07_DFF_PR



```
always @(negedge Reset or negedge Preset or posedge Clock)
begin
    if (!Reset) // Reset = 0
        Q = 1'b0; // 清除為 0
    else
        if (!Preset) // Preset = 0
            Q = 1'b1; // 設定為 1
        else
            if (Clock) // Reset = 1
                Q = D; // 將 D 傳入 Q
end
```

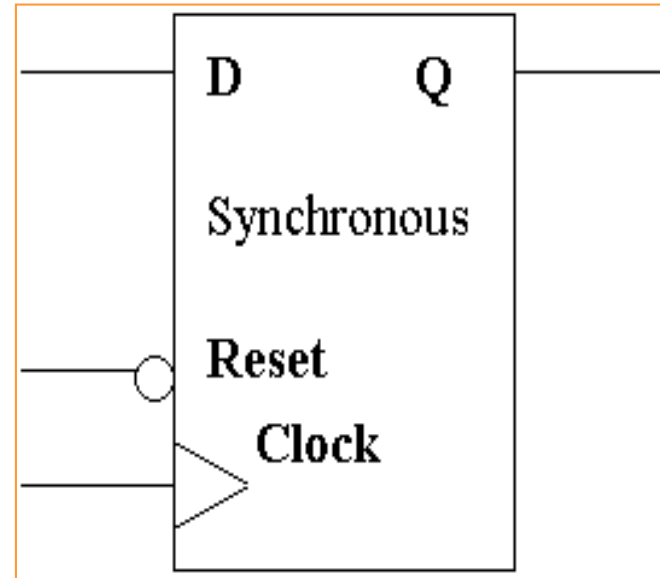
D-type Flip-flop (6)

範例6: 具有 **同步重置**(Synchronous Reset)的 D 型正反器

```
module D_FF_SR (Clock, Reset, D, Q);  
input  Clock, Reset, D;  
output Q;  
reg    Q;
```

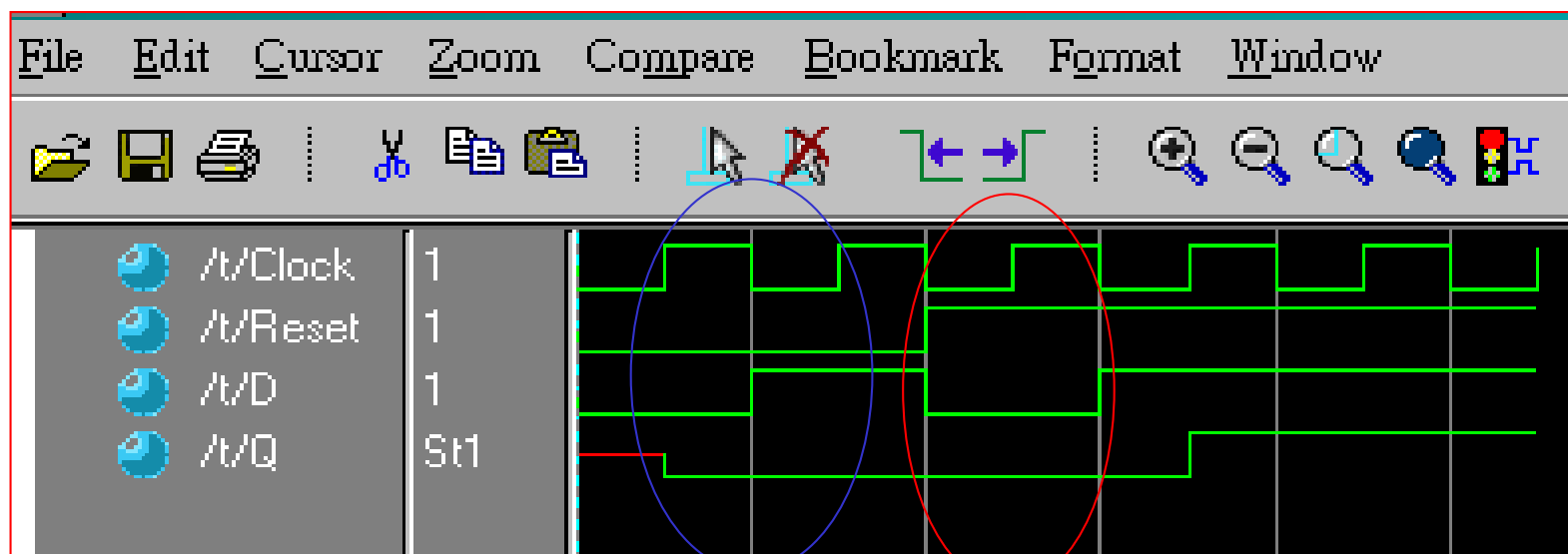
同步用法

```
    always @(posedge Clock)  
    begin  
        if (!Reset)  
            Q = 1'b0; // 清除為 0  
        else  
            Q = D; // 將 D 傳入 Q  
        end  
    endmodule
```



Simulation Waveform

4_08_DFF_SR



```
always @(posedge Clock)
begin
    if (!Reset) // Reset=0
        Q = 1'b0; // 清除為 0
    else
        Q = D; // 將 D 傳入 Q
end
```

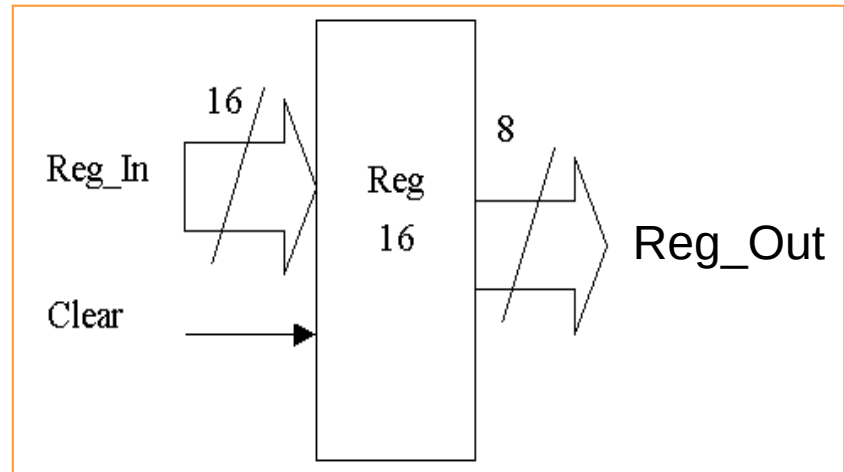
Register (1)

- ❑ **暫存器**：暫存器是一群二進位存取單元的集合，可用來儲存二進位資料。每一個存取單元通常為一個正反器，因此可以存取一個位元的資料，對於一個具有 n 個位元的暫存器，共可存取 n 個位元資料。
- ❑ **範例**：具有「清除」訊號的16位元暫存器

```
module Reg_16 (Clear, Reg_In, Reg_Out);  
  input  Clear;  
  input  [16:1] Reg_In;  
  output [16:1] Reg_Out;  
  reg    [16:1] Reg_Out;  
  always @(Clear or Reg_In)  
  begin  
    case (Clear)  
      0: Reg_Out[16:1] = 16'b0; // 清為 0  
      1: Reg_Out[16:1] = Reg_In[16:1]; // 載入資料  
    endcase  
  end  
endmodule
```

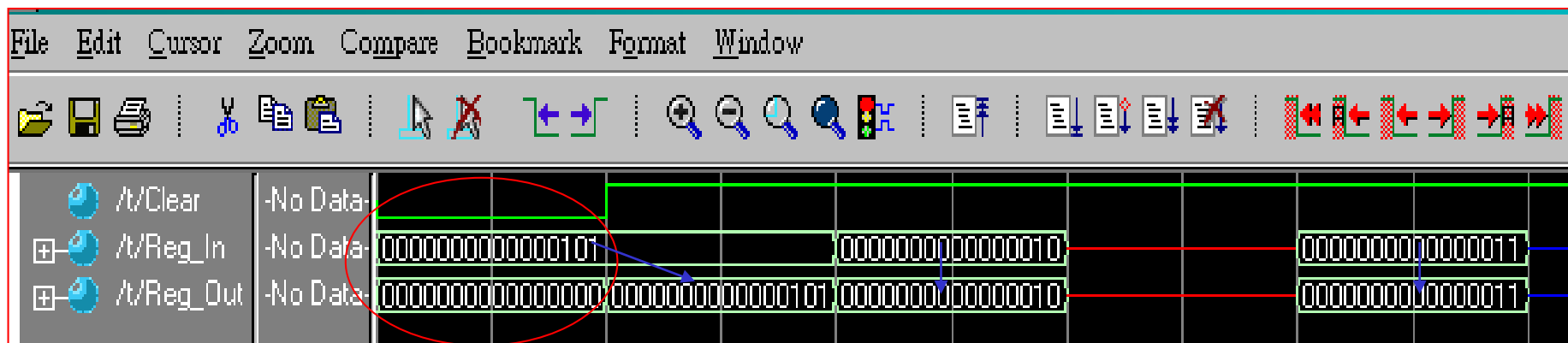
16-1+1
=16 bit

event list



Simulation Waveform

4_09_REG_16



```
always @(Clear or Reg_In)
begin
  case (Clear)
    0:
      Reg_Out[16:1] = 16'b0; // 清為 0
    1:
      Reg_Out[16:1] = Reg_In[16:1]; // 載入資料
  endcase
end
```

Register (2)

- ❑ 範例1：具「非同步負準位重置」與「時脈正緣觸發」的4位元暫存器
//RAS_NRPC.V，4位元暫存器

```
module RAS_NRPC (Reset, Clock, Reg_In, Reg_Out1);  
input    Reset, Clock;  
input    [3:0] Reg_In;  
output   [3:0] Reg_Out1;  
reg      [3:0] Reg_Out1;
```

```
always @(negedge Reset or posedge Clock)  
begin
```

```
    if (!Reset) // Reset = 0  
        Reg_Out1 = 0; // 清除為 0
```

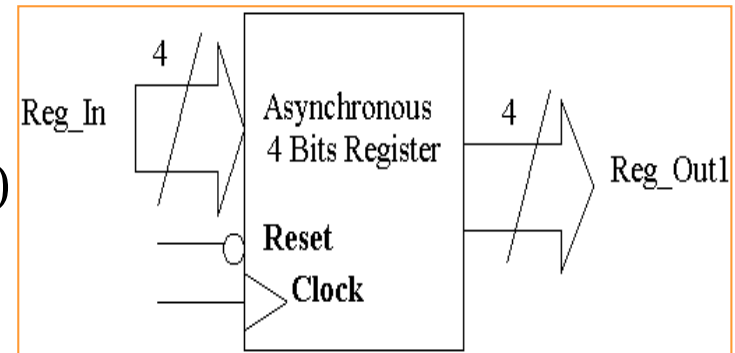
```
    else
```

```
        Reg_Out1 = Reg_In; // 將 Reg_In 傳入 Reg_Out1
```

```
    end
```

```
endmodule
```

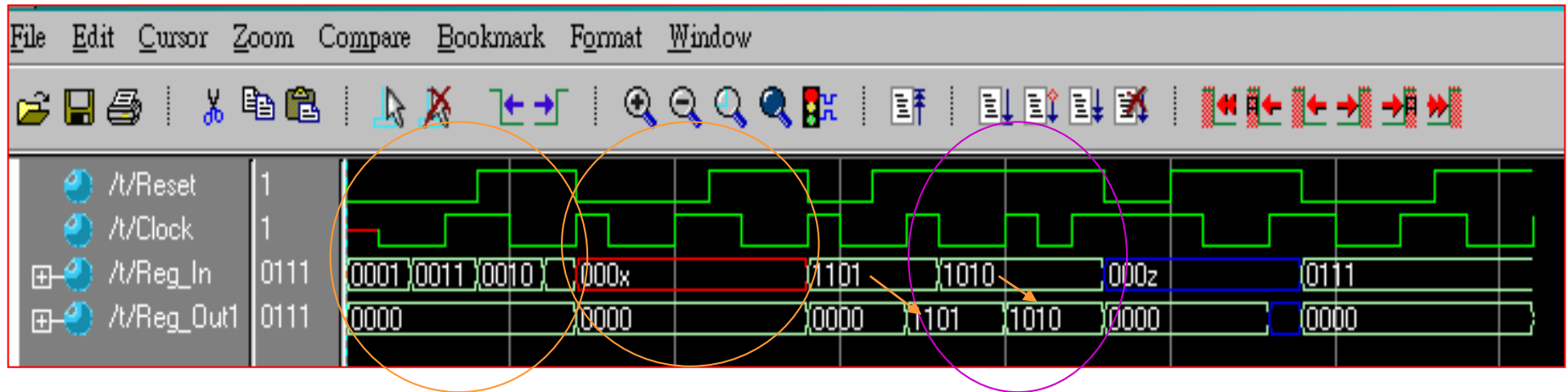
非同步負緣重置



Reset = 1 且 Clock = posedge

Simulation Waveform

4_10_RAS_NRPC



```
always @(negedge Reset or posedge Clock)
begin
    if (!Reset)           // Reset = 0
        Reg_Out1 = 0; // 清除為 0
    else
        Reg_Out1 = Reg_In; // 將 Reg_In 傳入 Reg_Out1
end
```

Register (3)

- ❑ 範例2：「同步正準位重置」與「時脈正緣觸發」訊號控制的4位元暫存器

```
module RS_NRPC (Reset, Clock, Reg_In, Reg_Out3);  
input    Reset, Clock;  
input    [3:0] Reg_In;  
output   [3:0] Reg_Out3;  
reg      [3:0] Reg_Out3;
```

```
always @(posedge Clock)  
begin
```

```
    if (!Reset) // Reset = 0
```

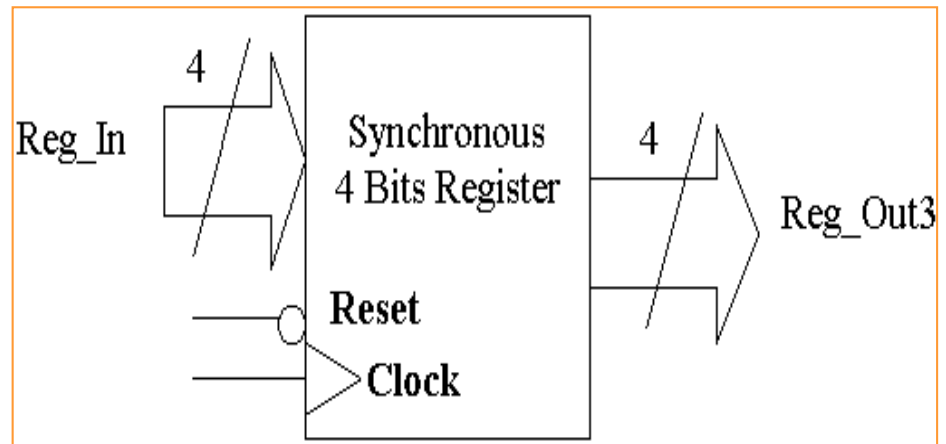
```
        Reg_Out3 = 0; // 清除為 0
```

```
    else
```

```
        Reg_Out3 = Reg_In; // 將 Reg_In 傳入 Reg_Out3
```

```
    end
```

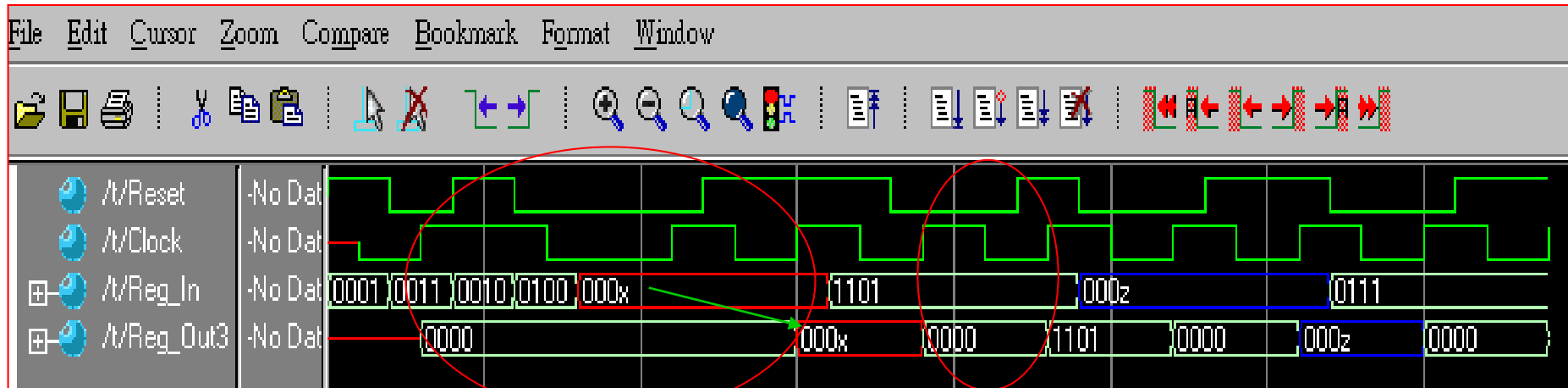
```
endmodule
```



在posedge Clock時
才執行begin...end內的敘述

Simulation Waveform

4_11_RS_NRPC

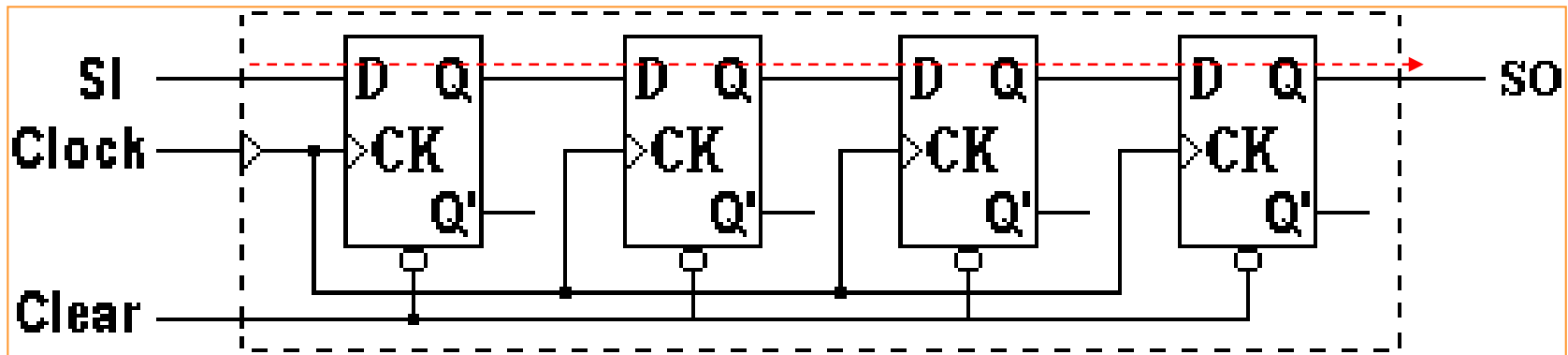


```
always @(posedge Clock)
begin
    if (!Reset)           // Reset = 0
        Reg_Out3 = 0; // 清除為 0
    else
        Reg_Out3 = Reg_In; // 將 Reg_In 傳入 Reg_Out3
end
```

Shift Register

2022/11/17

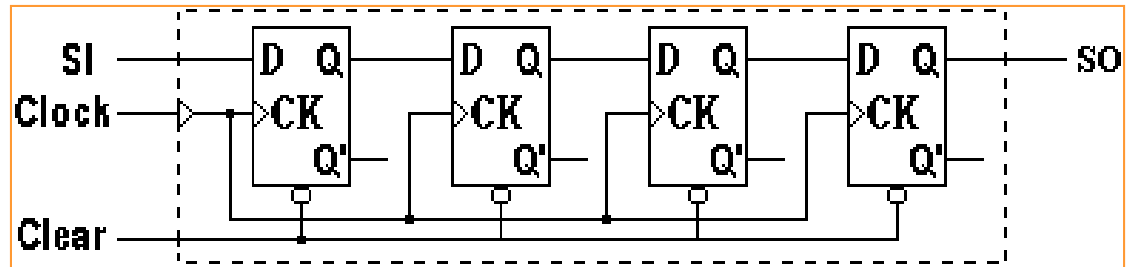
- **移位暫存器的功能**：資料格式的轉換、計數器、時序產生器 (Clock Generator) ... 等等，最簡單的移位暫存器是由一群 D 型正反器串接而成，以一個共同的時脈 (Clock Pulse) 訊號來驅動所有的正反器。
- **移位暫存器的功能**：其除了可以儲存資料以外，也可將儲存的資料向左或向右移動，故一個具有 n 個位元的移位暫存器通常是由 n 個 D 型正反器連接而成。
- **串進串出 (Serial In Serial Out) 移位暫存器**：其係用一個串列輸入 (Serial Input, SI) 埠用來決定移位期間輸入到最左端那個暫存器的值，串列輸出 (Serial Output, SO) 埠是用來輸出在這次的時脈 (Clock) 訊號來臨之前與上次時脈訊號來臨之間被儲存在最右邊的那個暫存器的值。



Serial In Serial Out (SISO) (1)

❑ 範例1：4位元串進串出，左移，移位暫存器

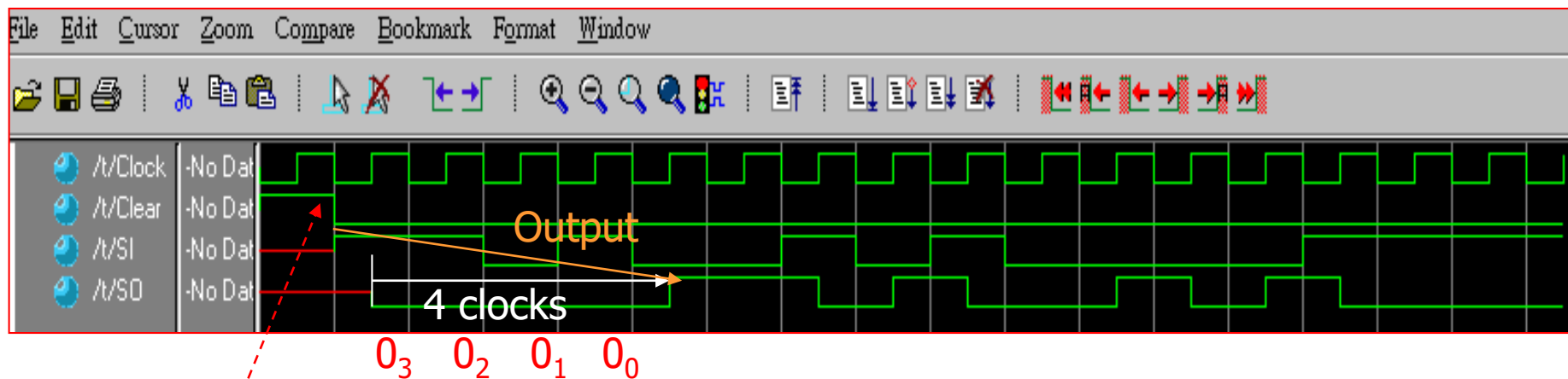
```
module SISO_Shift_Register (Clock, Clear, SI, SO);  
input  Clock, Clear, SI;  
output SO;  
reg    SO;  
reg    [3:0] Reg4;  
always @(posedge Clock)  
begin  
    if (Clear)      // Clear=1, 清除  
        Reg4 = 4'b0; // 清為 0  
    else  
    begin  
        SO = Reg4[3];  
        Reg4[3] = Reg4[2];  
        Reg4[2] = Reg4[1];  
        Reg4[1] = Reg4[0];  
        Reg4[0] = SI; // 移入新的資料 (SI) 至 Reg4[0]  
    end  
end  
endmodule
```



順序更改，結果如何？

Simulation Waveform (1)

4_12_SISO



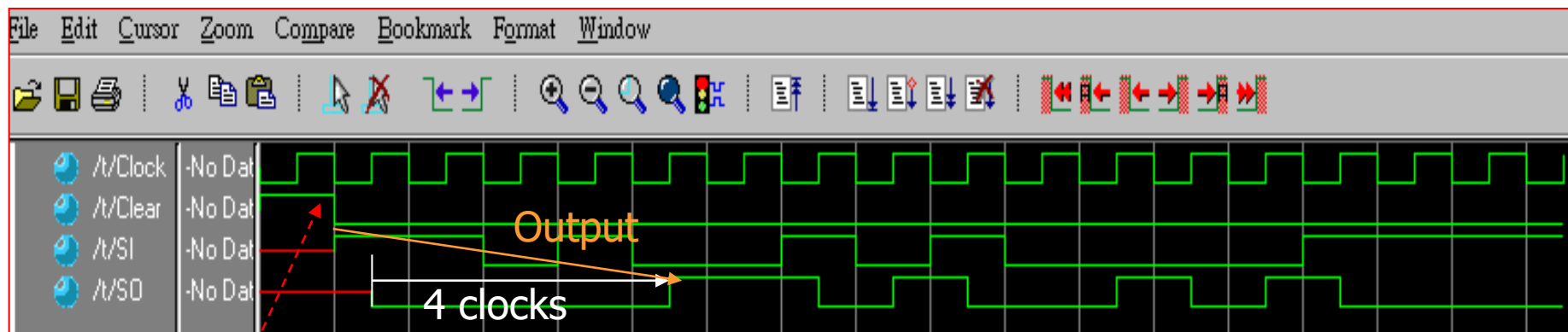
Reg4=[0000]

```
always @(posedge Clock)
begin
    if (Clear) // Clear=1, 清除
        Reg4 = 4'b0; // 清為 0
    else
    begin
        SO = Reg4[3];
        Reg4[3] = Reg4[2];
        Reg4[2] = Reg4[1];
        Reg4[1] = Reg4[0];
        Reg4[0] = SI; // 移入新的資料 (SI) 至 Reg4[0]
    end
end
```

請改寫程式把Reg4的值顯示出來。
=>testbench要加入 .Reg4(Reg4)

Simulation Waveform (2)

4_12_SISO_1



```
always @(posedge Clock)
```

```
begin
```

```
if (Clear) // Clear=1, 清除
```

```
Reg4 = 4'b0; // 清為 0
```

```
else
```

```
begin
```

```
SO = Reg4[3];
```

```
Reg4[3] = Reg4[2];
```

```
Reg4[2] = Reg4[1];
```

```
Reg4[1] = Reg4[0];
```

```
Reg4[0] = SI; // 移入新的資料 (SI) 至 Reg4[0]
```

```
end
```

```
end
```

請改寫程式把[3:0]Reg4的值顯示出來。

=>testbench要加入

wire [3:0] Reg4;

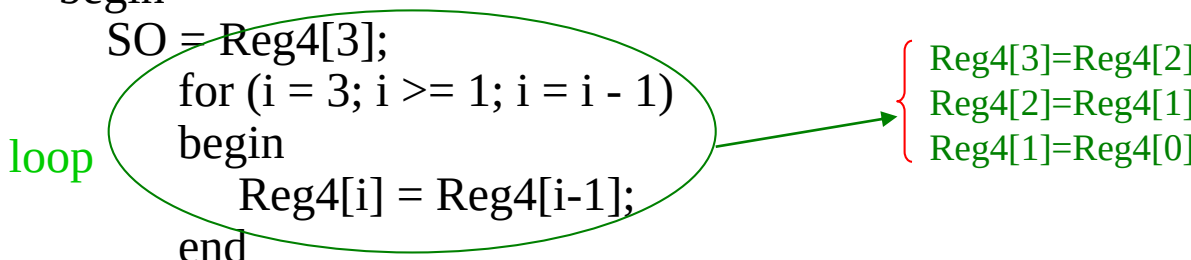
...

.Reg4(Reg4)

Serial In Serial Out (SISO) (2)

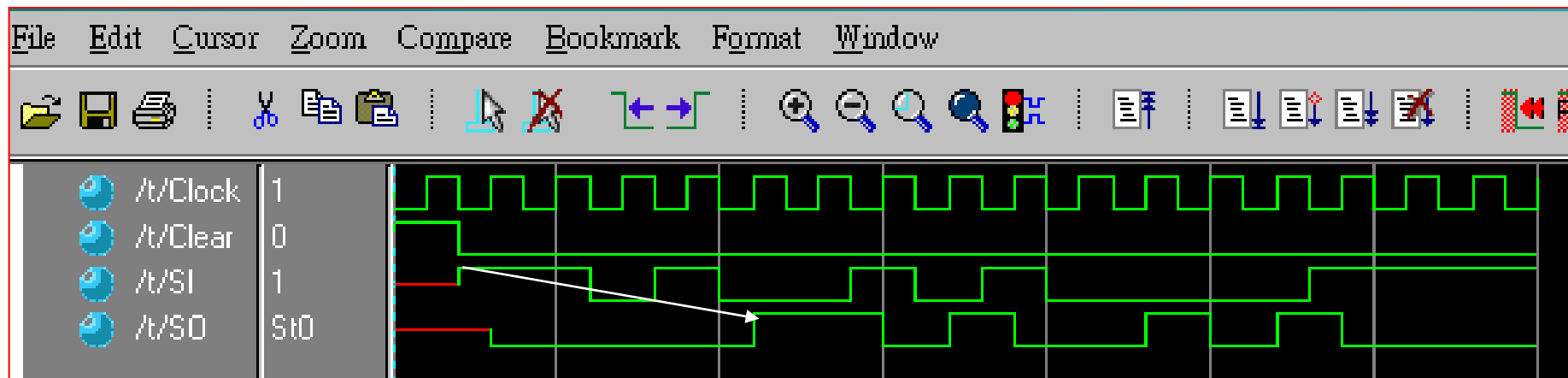
□ 範例2: 4位元串進串出，左移，移位暫存器(使用 for 敘述, 來設計)

```
module SISO_Shift_Register (Clock, Clear, SI, SO);
input  Clock, Clear, SI;
output SO;
reg    SO;
reg    [3:0] Reg4;
always @(posedge Clock)
begin : for_Local // 定義一個區域名稱 for_Local
    integer i;      // 定義一個區域性變數
    if (Clear)      // 清除
        Reg4 = 4'b0; // 清為 0
    else
    begin
        SO = Reg4[3];
        for (i = 3; i >= 1; i = i - 1)
        begin
            loop Reg4[i] = Reg4[i-1];
        end
        Reg4[0] = SI; // 移入新的資料 (SI) 至 Reg4[0]
    end
end
end
endmodule
```



Simulation Waveform

4_13_SISO_FOR



```
always @(posedge Clock)
begin : for_Local // 定義一個區域名稱 for_Local
integer i; // 定義一個區域性變數
if (Clear) // 清除
Reg4 = 4'b0; // 清為 0
else
begin
SO = Reg4[3];
for (i = 3; i >= 1; i = i - 1)
begin
Reg4[i] = Reg4[i-1];
end
Reg4[0] = SI; // 移入新的資料 (SI) 至 Reg4[0]
end
end
```

Serial In Parallel Out (SIPO)

❑ 串進並出 (Serial In Parallel Out) 移位暫存器：其用一個串列輸入(Serial Input, SI)埠來決定移位期間輸入到最左端暫存器的值，並列輸出 (Parallel Output, PO)埠用來輸出在這次的時脈(Clock)訊號動作之後，儲存在n個位元移位暫存器內的值。

❑ 範例：4位元串進並出左移暫存器

```
module SIPO_Shift_Register (Clock, Clear, SI, Data_Out);  
input    Clock, Clear, SI; Parallel Out  
output   [3:0] Data_Out;  
reg      [3:0] Data_Out;
```

```
always @(posedge Clock)
```

```
begin
```

```
    if (Clear) // 清除
```

```
        Data_Out = 4'b0; // 清為 0
```

```
    else
```

```
        begin
```

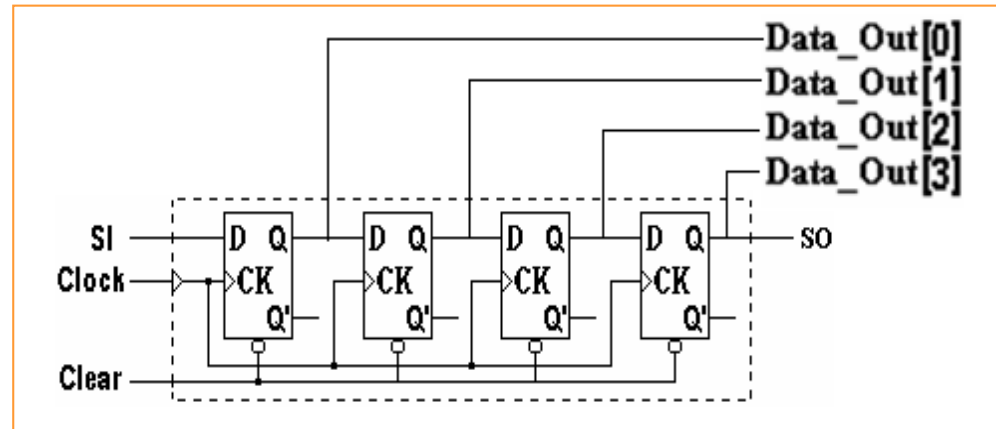
```
            Data_Out = Data_Out << 1; // 左移 1 位元
```

```
            Data_Out [0] = SI; // 移入新的資料 (SI) 至 Data_Out [0]
```

```
        end
```

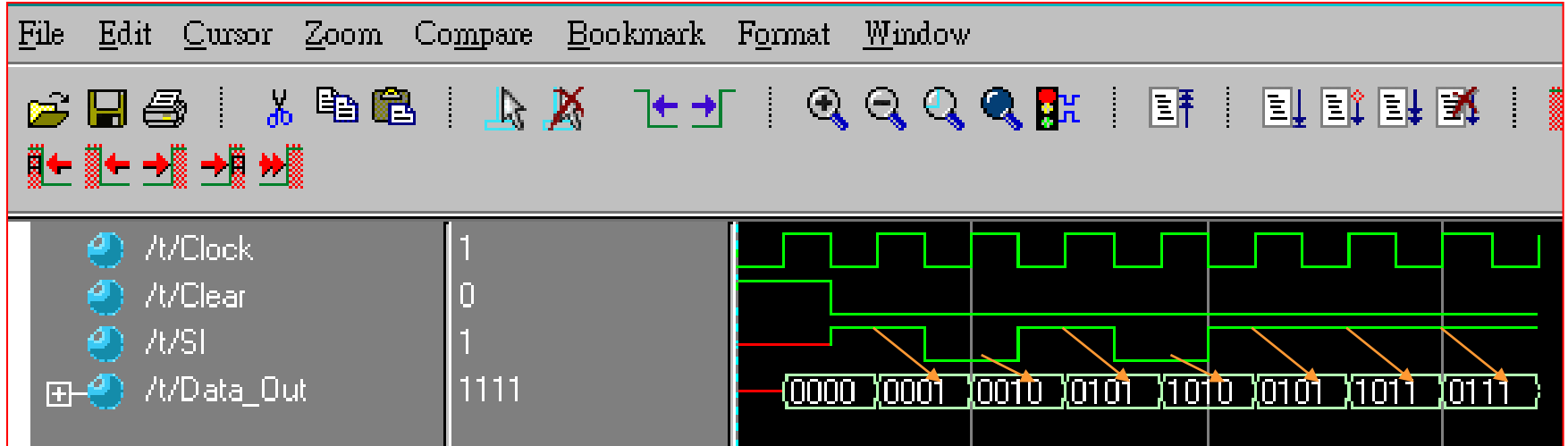
```
    end
```

```
endmodule
```



Simulation Waveform

4_14_SIPO



```
always @(posedge Clock)
```

```
begin
```

```
if (Clear) // 清除
```

```
    Data_Out = 4'b0; // 清為 0
```

```
else
```

```
begin
```

```
    Data_Out = Data_Out << 1; // 左移 1 位元
```

```
    Data_Out [0] = SI; // 移入新的資料 (SI) 至 Data_Out [0]
```

```
end
```

```
end
```

先左移再串入後並出 => 請改寫為”先並出再串進”。

Appendix: Serial In Parallel Out (SIPO)

4_14_SIPO_1

□ 範例：4位元串進並出左移暫存器

```
module SIPO_Shift_Register (Clock, Clear, SI, Data_Out);
```

```
input    Clock, Clear, SI;
```

```
output   [3:0] Data_Out;
```

```
reg      [3:0] Data_Out;
```

```
reg      [3:0]temp;
```

```
always @(posedge Clock)
begin
```

```
temp = 4'b0; //initial;
```

```
...
```

```
Data_Out = temp;
```

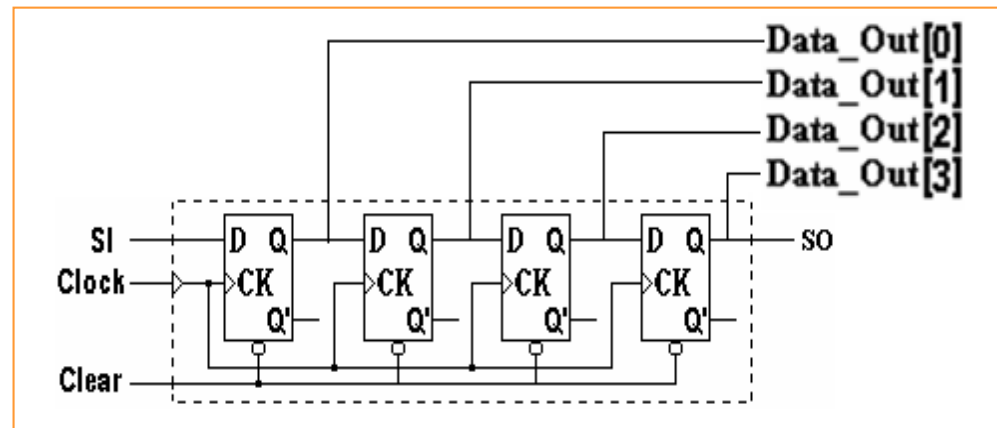
```
temp = temp << 1; // 左移 1 位元
```

```
temp[0] = SI; // 移入新的資料 (SI) 至Data_Out [0]
```

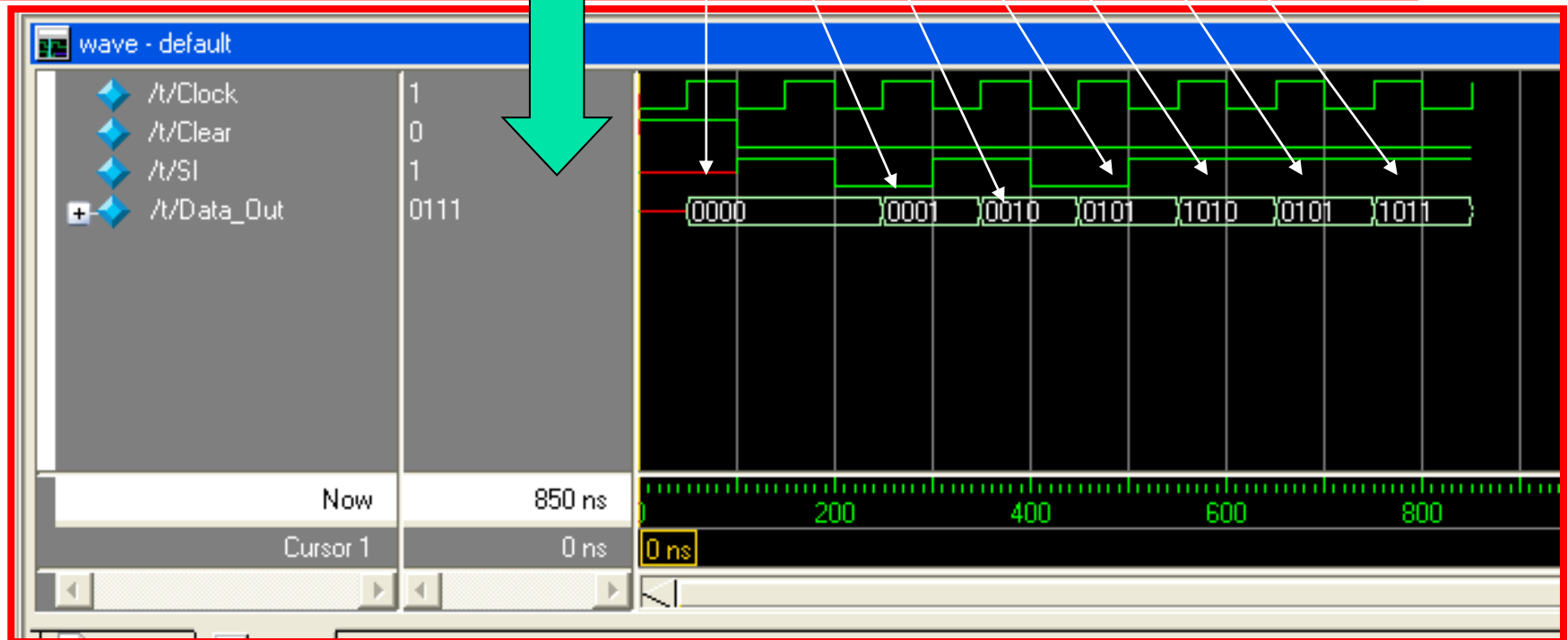
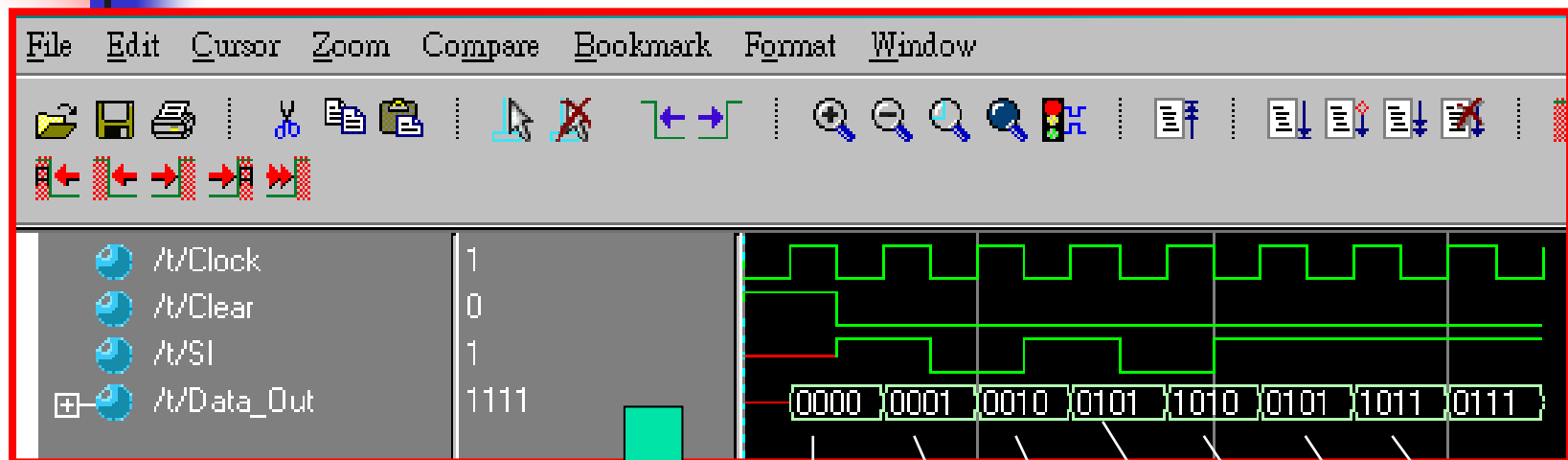
```
...
```

```
end
```

```
endmodule
```

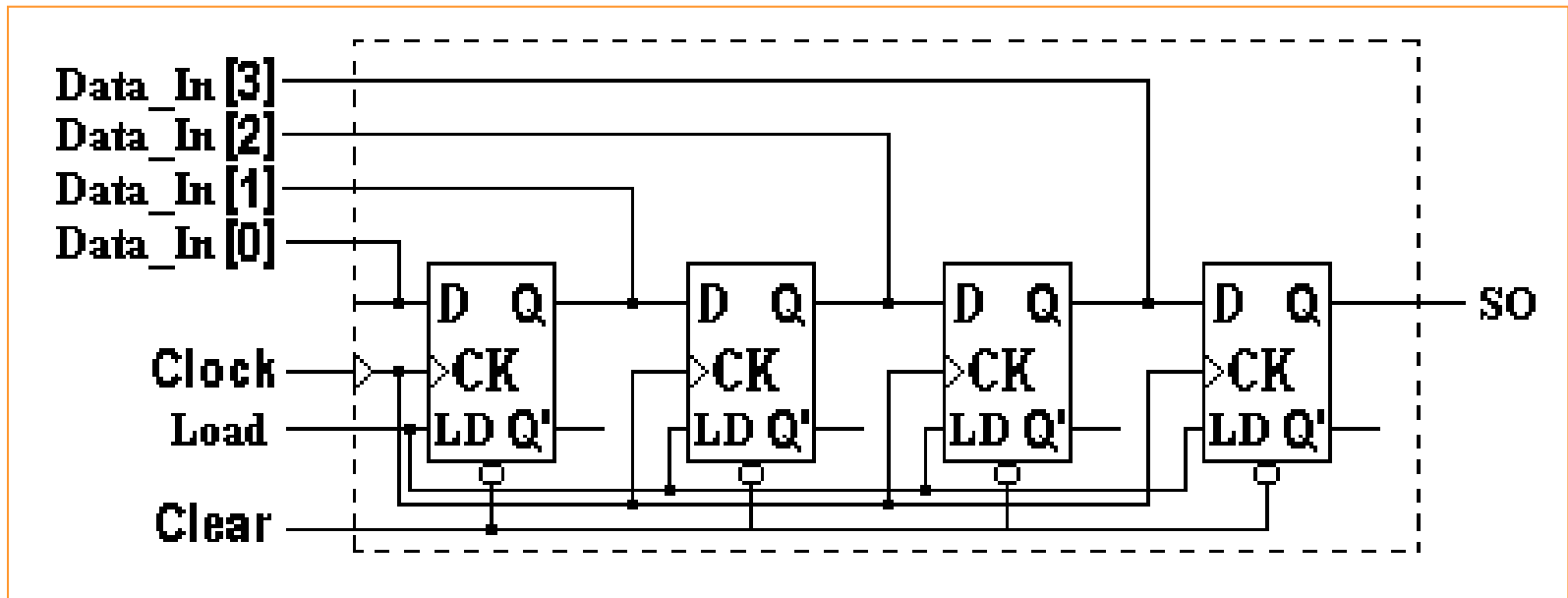


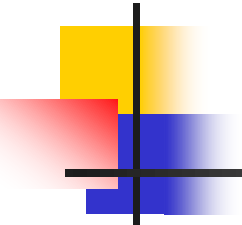
Appendix: Serial In Parallel Out (SIPO)



Parallel In Serial Out (PISO) (1)

- **並進串出 (Parallel In Serial Out) 移位暫存器**：當時脈 (Clock) 訊號及載入 (Load) 訊號皆動作時，會將並列輸入 (Parallel Input, PI) 埠上的資料傳送到移位暫存器的 n 個暫存器之中；**串列輸出 (Serial Output, SO)** 埠是用來輸出，即當時脈 (Clock) 訊號動作且載入 (Load) 訊號不動作(0)的情況下，送出最右邊的那個暫存器的值。





Parallel In Serial Out (PISO) (2)

//PISO.V：4位元並進串出，左移，移位暫存器

```
module PISO_Shift_Register (Clock, Clear, Load, Data_In, SO);
```

```
input    Clock, Clear, Load;
```

```
input    [3:0] Data_In;
```

```
output    SO;
```

```
reg       SO;
```

```
reg       [3:0] Reg4;
```

```
always @(posedge Clock)
```

```
begin : for_Local    // 定義一個區域名稱 for_Local
```

```
    integer i;        // 定義一個區域性變數
```

```
    if (Clear)        // 清除
```

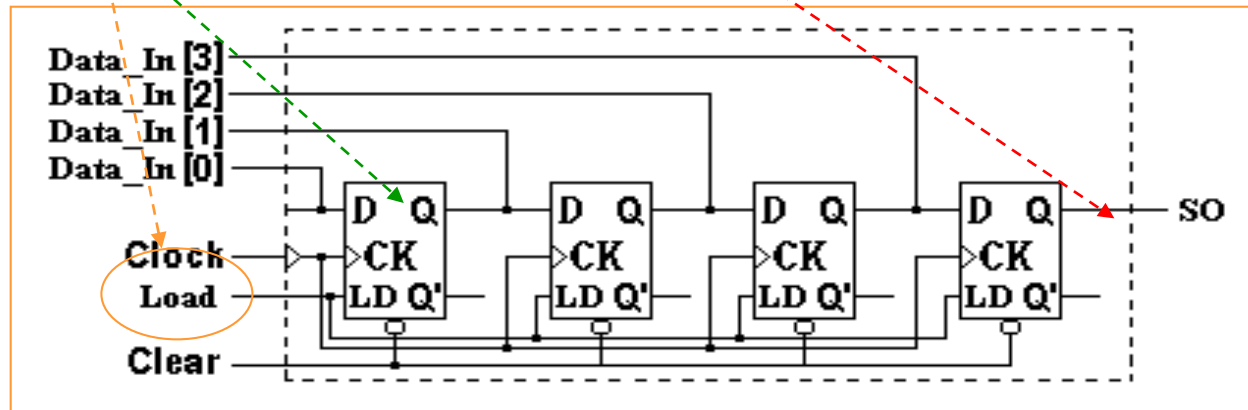
```
        Reg4 = 4'b0; // 清除為 0
```

```
    else
```

```
    begin
```

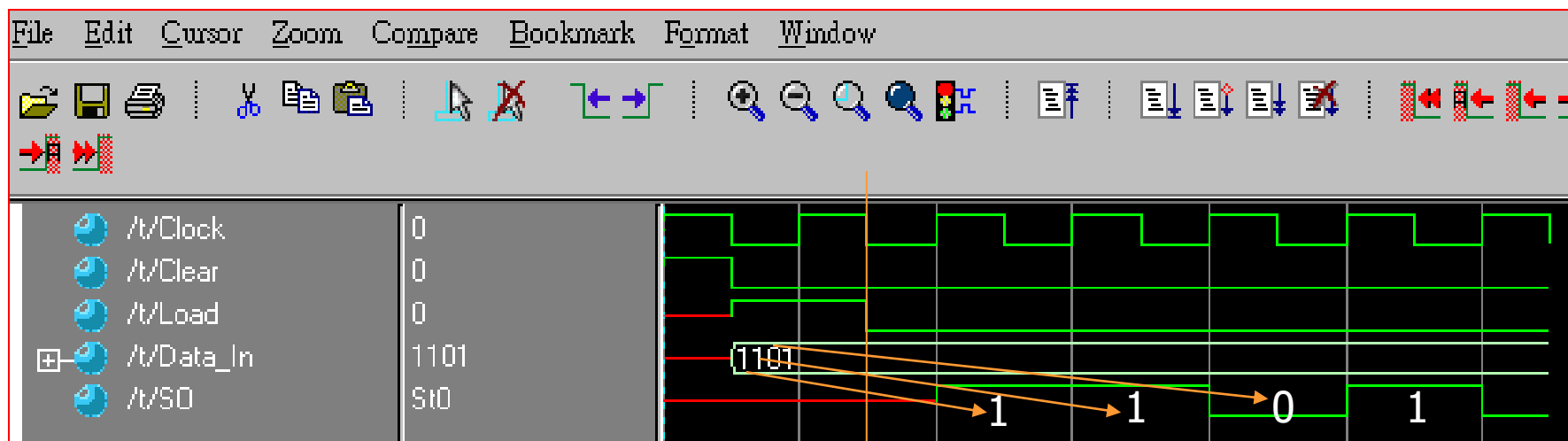
Parallel In Serial Out (PISO) (3)

```
if(Load)
    Reg4 = Data_In; // 從 Data_In[3:0] 載入資料
else
begin
    SO = Reg4[3];
    for (i = 3; i >= 1; i = i - 1)
    begin
        Reg4[i] = Reg4[i-1];
    end
    Reg4[0] = 0; // Reg4[0] 填入 0
end
end
end
endmodule
```



Simulation Waveform

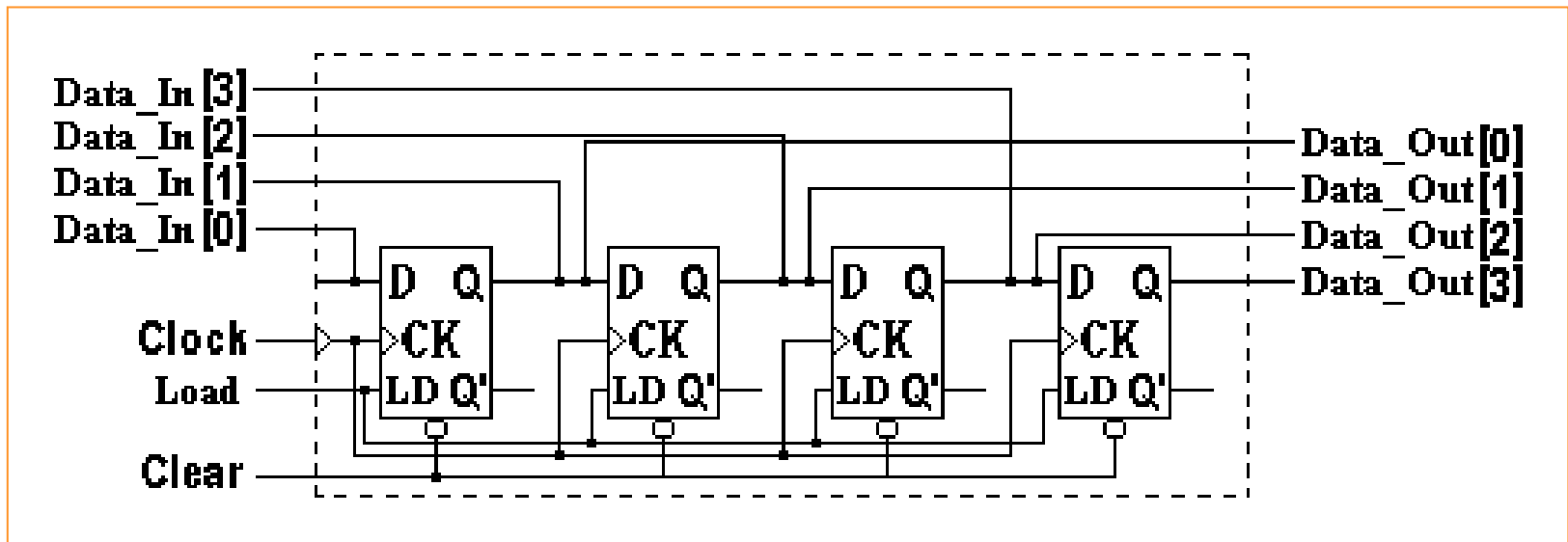
4_15_PISO



```
if(Load)
    Reg4 = Data_In; // 從 Data_In 載入資料
else
    begin
        SO = Reg4[3];
        for (i = 3; i >= 1; i = i - 1)
            begin
                Reg4[i] = Reg4[i-1];
            end
        Reg4[0] = 0; // Reg4[0] 填入 0
    end
```

Parallel In Parallel Out (PIPO) (1)

- 並進並出 (Parallel In Parallel Out) 暫存器：當時脈 (Clock) 訊號及載入 (Load=1) 訊號皆動作時，會將並列輸入 (Parallel Input, PI) 埠上的資料傳送到移位暫存器的 n 個暫存器之中，同時將儲存在 n 個位元移位暫存器內的值輸出（意義不大，可修正）。



Parallel In Parallel Out (PIPO)

//PIPO.V : 4位元並進並出移位暫存器

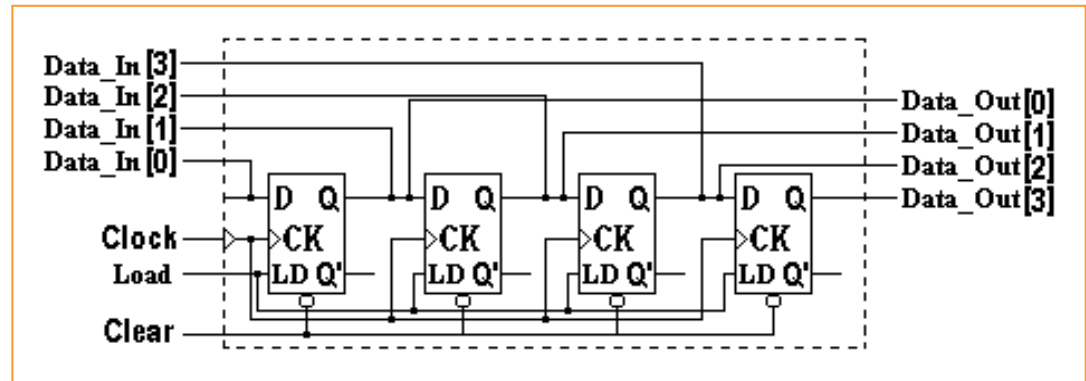
```
module PIPO_Shift_Register(Clock, Clear, Load, Data_In, Data_Out);
```

```
input Clock, Clear, Load;
```

```
input [3:0] Data_In;
```

```
output [3:0] Data_Out;
```

```
reg [3:0] Data_Out;
```



```
always @(posedge Clock)
```

```
begin
```

```
if (Clear)
```

```
    Data_Out = 4'b0; // 清除 // 清為 0
```

```
else
```

```
begin
```

```
    if(Load)
```

```
        Data_Out = Data_In; // 從 Data_In 載入資料
```

```
    end
```

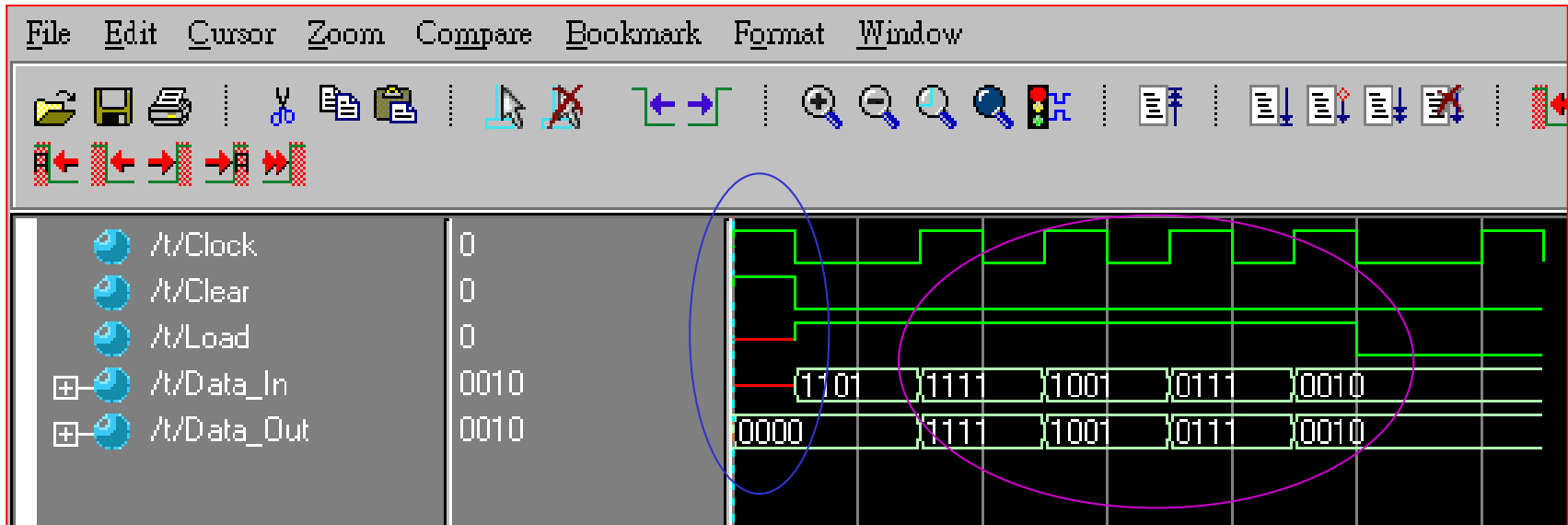
```
end
```

```
endmodule
```

有誤？請修正(加入temp)。

Simulation Waveform

4_16_PIPO



```
always @(posedge Clock)
begin
  if (Clear) // 清除
    Data_Out = 4'b0; // 清為 0
  else
    begin
      if(Load)
        Data_Out = Data_In; // 從 Data_In 載入資料
    end
end
```

建議修正：

- (1). 並列輸出：當時脈 (Clock) 訊號為負緣觸發時，將儲存在 n 個位元移位暫存器內的值輸出。
- (2). 並列輸出：當時脈 (Clock) 訊號不動作且載入 (Load=0) 訊號不動作的情況下，將儲存在 n 個位元移位暫存器內的值輸出。

PIPO Modification (1)

4_16_PIPO_new01

//PIPO.V : 4位元並進並出移位暫存器

```
module PIPO_Shift_Register_new(Clock, Clear, Load, Data_In, Data_Out);
```

```
input Clock, Clear, Load;
```

```
input [3:0] Data_In;
```

```
output [3:0] Data_Out;
```

```
reg [3:0] Data_Out;
```

```
reg [3:0]temp;
```

```
always @(posedge Clock)
```

```
begin
```

```
if (Clear) // 清除
```

```
Data_Out = 4'b0; // 清為 0
```

```
else
```

```
begin
```

```
if(Load)
```

```
begin
```

```
Data_Out <= temp; // 從 Data_In 載入資料
```

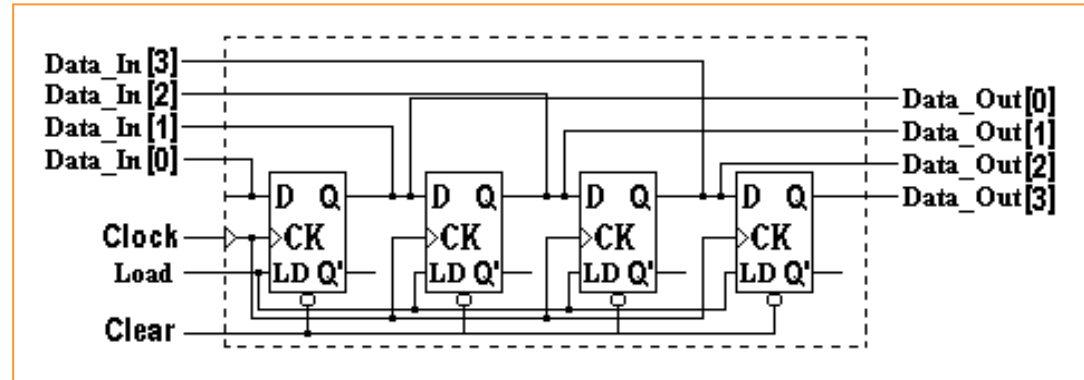
```
temp <= Data_In;
```

```
end
```

```
end
```

```
end
```

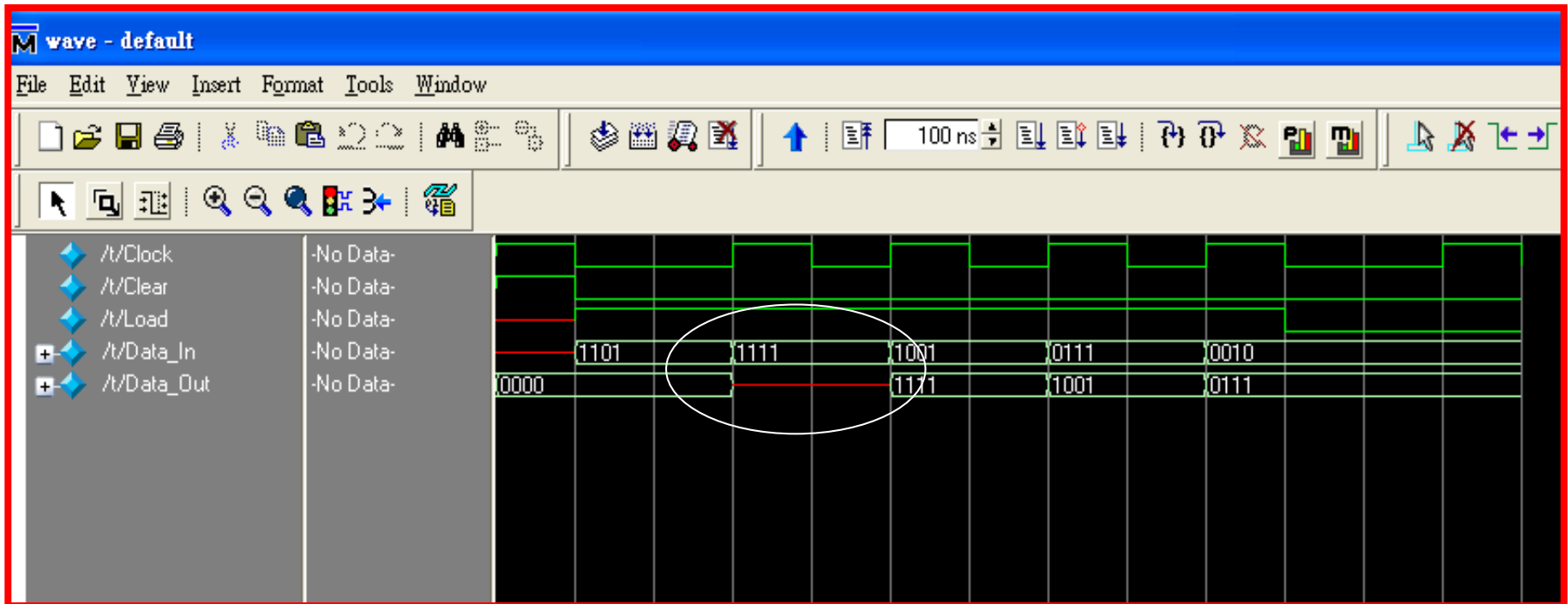
```
endmodule
```



第一筆資料不見了,請修正程式。

PIPO Modification (2)

4_16_PIPO_new01



PIPO Modification (3)

4_16_PIPO_new02

//PIPO.V : 4位元並進並出移位暫存器

```
module PIPO_Shift_Register_new(Clock, Clear, Load, Data_In, Data_Out);
```

```
input Clock, Clear, Load;
```

```
input [3:0] Data_In;
```

```
output [3:0] Data_Out;
```

```
reg [3:0] Data_Out;
```

```
wire [3:0] temp;
```

(1) **assign temp = Data_In;**

```
always @(posedge Clock)
```

```
begin
```

(2)

```
if (Clear) // 清除
```

```
    Data_Out = 4'b0; // 清為 0
```

```
else
```

```
begin
```

```
    if(Load)
```

```
        begin
```

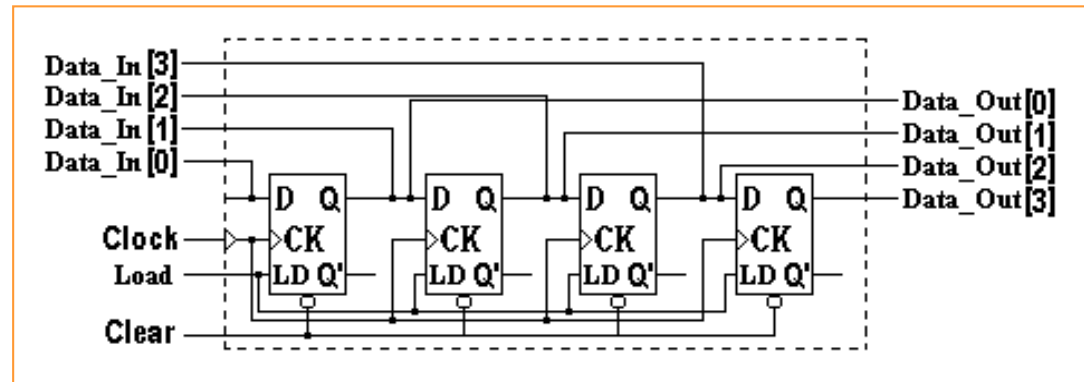
Data_Out <= temp; // 從 Data_In 載入資料

```
        end
```

```
    end
```

```
end
```

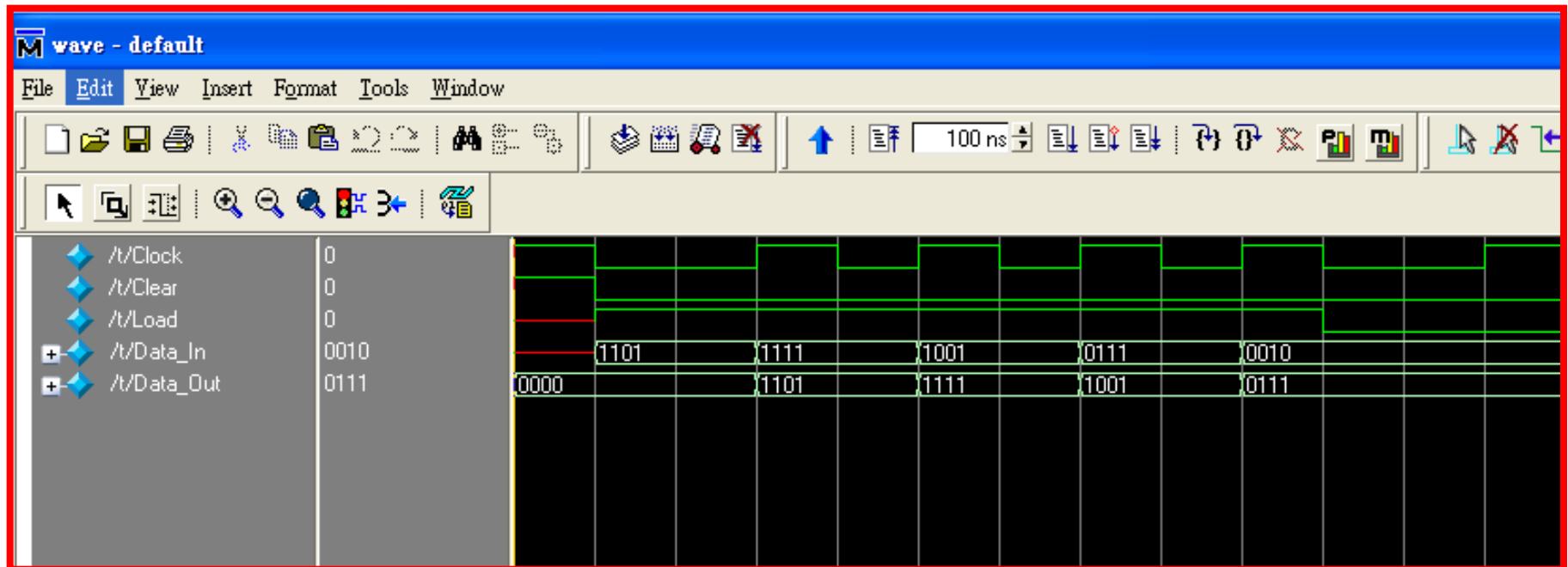
```
endmodule
```



Wire [3:0] → Reg [3:0]

PIPO Modification (4)

4_16_PIPO_new02



Counter (1)

- ❑ **上數計數器(Count Up Counter)：**「上數計數器」又稱為「正數計數器」。在非同步的上數計數器的內部中，其後級的時脈訊號是由前一級的非補數輸出端取得的。
 - 以「3 個位元的上數計數器」而言，它的計數範圍由 0、1、2 ... 到 6、7，再回到 0、1、2 ... 等循環計數的方式。
- ❑ **下數計數器(Count Down Counter)：**「下數計數器」又稱為「倒數計數器」。在非同步的上數計數器的內部中，其後級的時脈訊號是由前一級的補數輸出端取得的。
 - 以 3 個位元的下數計數器而言，它的計數範圍由 7、6、5 ... 到 1、0，再回到 7、6、5 ... 等循環計數的方式。
- ❑ **上數/下數計數器 (Up/Down Counter)：**「上數/下數計數器」又稱為「正/倒數計數器」，它具有上數及下數的功能，通常用一個「上/下數」(Up/Down) 的控制訊號來決定電路是要作上數或下數的計數。
 - 在一個上數/下數計數器的內部中，可以用一個2對1的多工器選擇正反器。

Counter (2)

//Counter2.V：上數計數器

//具「重置」、「載入」及「致能輸出」控制的8位元正緣觸發上數計數器

```
module Counter2 (Clock, Reset, Load, Enable, Data_In, Out);
```

```
input Clock, Reset, Load, Enable;
```

```
input [7:0] Data_In;
```

```
output [7:0] Out;
```

```
reg [7:0] Out;
```

```
always @ (posedge Clock)
```

```
begin
```

```
if (Reset) // 重置
```

```
Out = 0; // 清除為 0
```

```
else
```

```
if (Load) // 載入
```

```
Out = Data_In;
```

```
// 從輸入埠 Data_In, 載入計數值
```

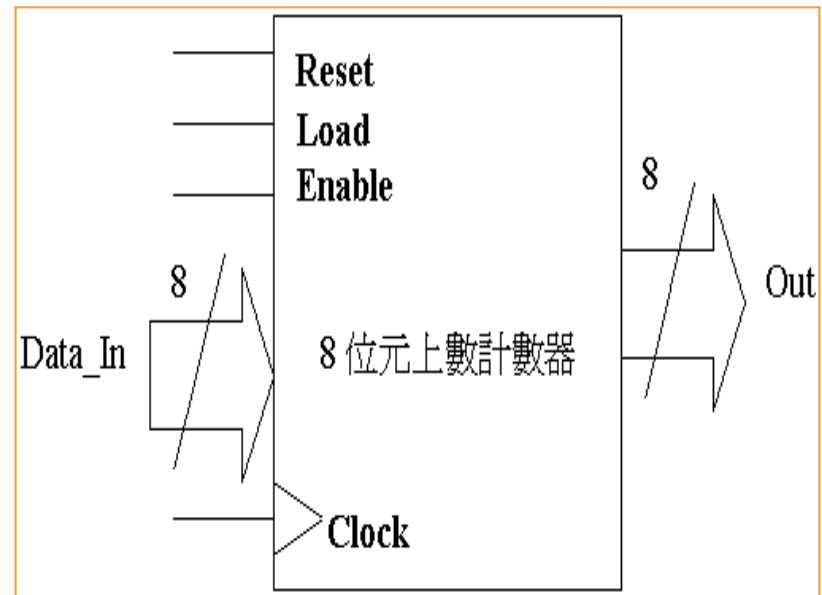
```
else
```

```
if (Enable) // 致能計數
```

```
Out = Out + 1; // 計數值 +1
```

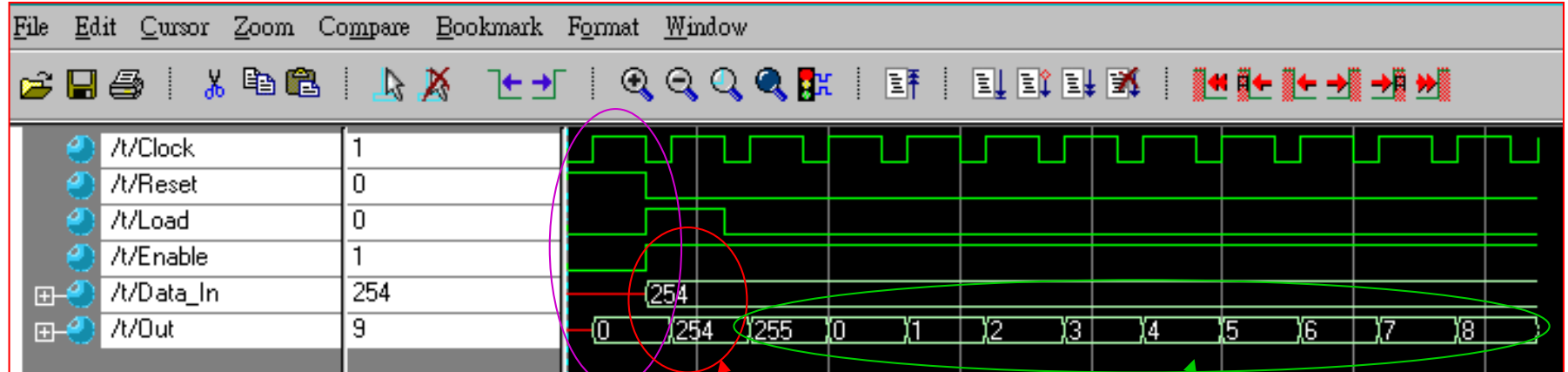
```
end
```

```
endmodule
```



Simulation Waveform

4_18_COUNTER2

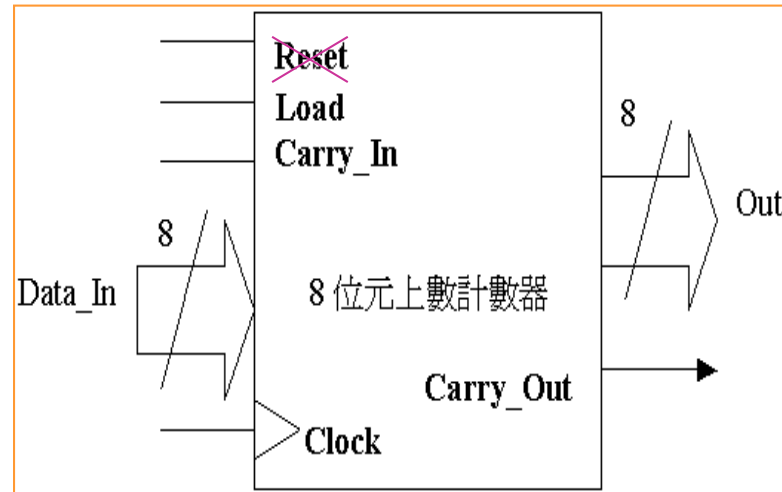


```
always @ (posedge Clock)
begin
    if (Reset) // 重置
        Out = 0; // 清除為 0
    else
        if (Load) // 載入
            Out = Data_In;
            // 從輸入埠 Data_In, 載入計數值
        else
            if (Enable) // 致能計數
                Out = Out + 1; // 計數值 +1
end
```

$2^8=256$ (0~255)

Counter (3)

```
//Counter3.V : 8位元 (0~255) 正緣觸發上數計數器
//具「重置」及「載入」訊號控制的8位元正緣觸發上數計數器
module Counter3 (Clock, Load, Carry_In, Data_In, Out, Carry_Out);
input      Clock, Load, Carry_In;
input      [7:0] Data_In;
output     [7:0] Out;
output     Carry_Out;
reg        [7:0] Out;
reg        Carry_Out;
reg      [7:0] Pre_Out;
```



// 產生一個 8 位元的暫存器

always @(posedge Clock) // 當Clock正緣觸發時

begin

Out = Pre_Out; // 更新 Out 值

end

→ 第一個Block，將Pre_Out的值傳至Out

Counter (4)

// 計算下一個計數器的狀態及 Carry_Out 的值

// 基於「效能」上的考量, 我們不希望 Load 訊號影響 Carry_Out
always @(Load or Data_In or Carry_In or Out) 僅影響Pre_Out

begin

{Carry_Out, Pre_Out} = Out + Carry_In;

if (Load) // 載入

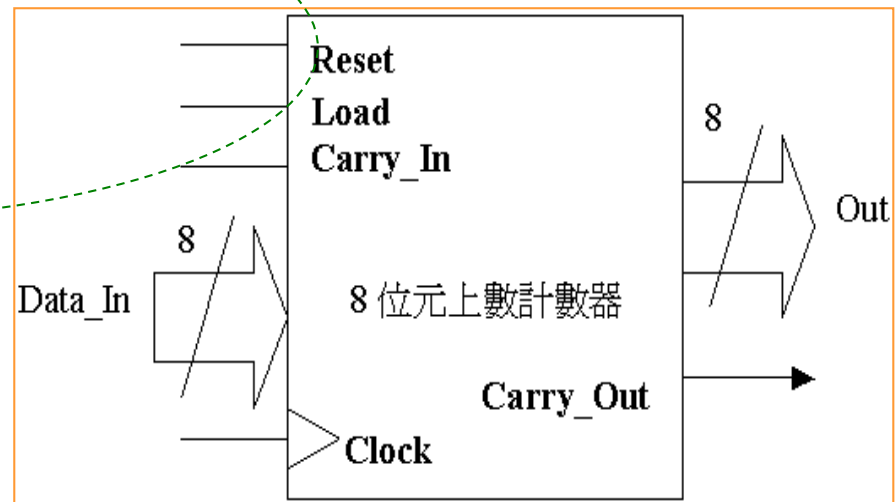
Pre_Out = Data_In;

// 從輸入埠 Data_In,

// 載入計數值到Pre_Out

end

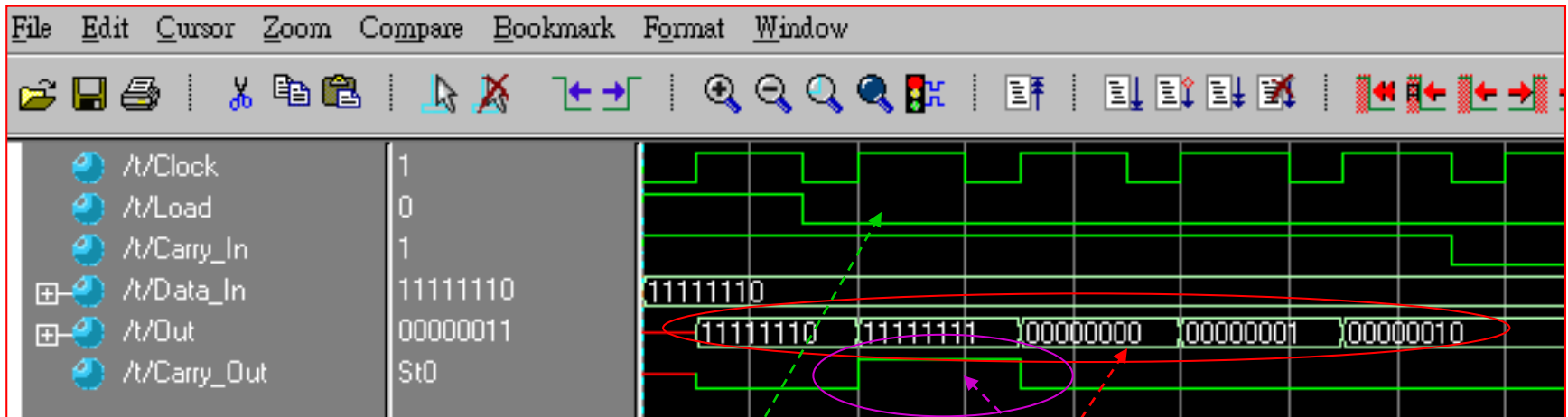
endmodule



第二個Block, 計算Carry_Out和Pre_Out的值

Simulation Waveform

4_19_COUNTER3



```
always @(posedge Clock) // 當Clock正緣觸發時
begin
    Out = Pre_Out; // 更新 Out 值
end
always @(Load or Data_In or Carry_In or Out)
begin
    {Carry_Out, Pre_Out} = Out + Carry_In;
    if (Load) // 載入
        Pre_Out = Data_In; // 從輸入埠 Data_In, 載入計數值到Pre_Out
end
endmodule
```

Wrong?

Counter (5)

//Counter4.V：10位元正緣觸發上數計數器

// 若計數值為0, 則 Zero = 1

// 若計數值為1023, 則 Max_1023 = 1

// 若計數值為偶同位, 則 Even = 1

// 若計數值為奇同位, 則 Odd = 1

```
module Counter4 (Clock, Reset, Zero, Max_1023, Even, Odd, Out);
```

```
input Clock, Reset;
```

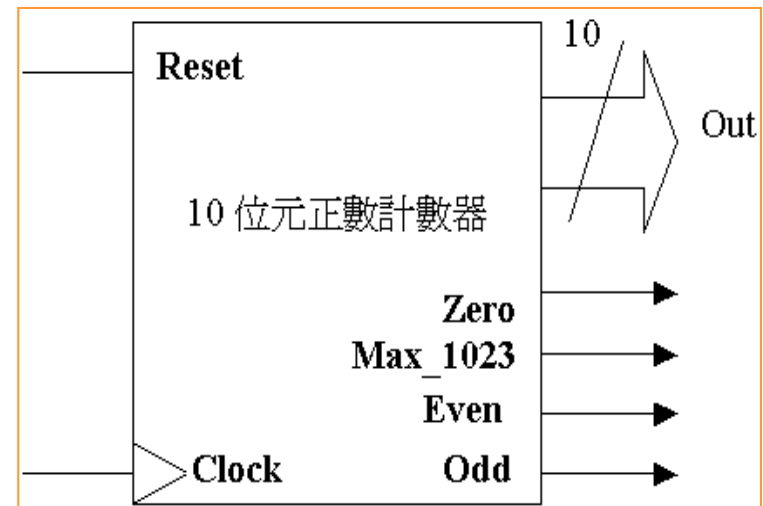
```
output Zero, Max_1023, Even, Odd;
```

```
Reg Zero, Max_1023, Even, Odd;
```

```
output [9:0] Out;
```

```
reg [9:0] Out;
```

將” Reg”修正為” wire”結果如何？



Counter (6)

always @(posedge Clock or posedge Reset)

begin

if (Reset) // 重置

Out = 0; // 清除為0

else

Out = Out + 1; // 計數值 加1

Zero = ! (| Out);

Max_1023 = & Out;

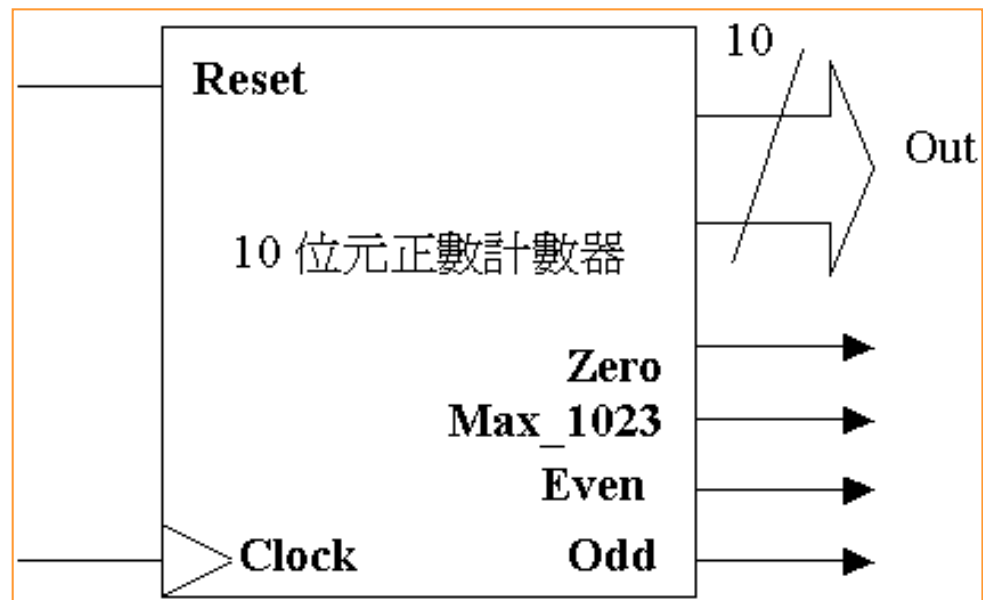
Odd = ^ Out;

Even = ! (Odd);

end

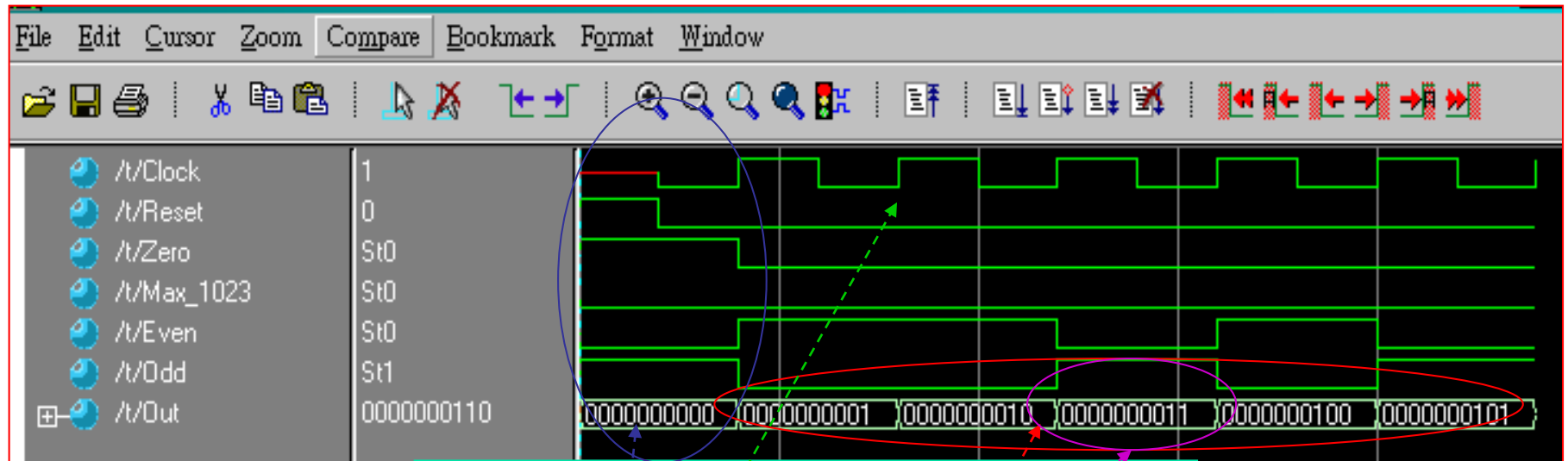
endmodule

若計數值為0, 則 Zero = 1
若計數值為1023, 則 Max_1023 = 1
若計數值為偶同位, 則 Even = 1
若計數值為奇同位, 則 Odd = 1



Simulation Waveform

4_20_COUNTER4



```
always @(posedge Clock or posedge Reset)
begin
    if (Reset) // 重置
        Out = 0; // 清除為0
    else
        Out = Out + 1; // 計數值 加1
    Zero = !(| Out);
    Max_1023 = & Out;
    Odd = ^ Out;
    Even = ! (Odd);
end
endmodule
```

XOR: 相同為0，不同為1.

Counter (7)

//Counter5.V：10位元正緣觸發上數計數器

// 若計數值為0, 則 Zero = 1

// 若計數值為1023, 則 Max_1023 = 1

// 若計數值為偶同位, 則 Even = 1

// 若計數值為奇同位, 則 Odd = 1

// 將判斷計數值的寫法, 置於 always 區塊外「效能」較佳

```
module Counter4 (Clock, Reset, Zero, Max_1023, Even, Odd, Out);
```

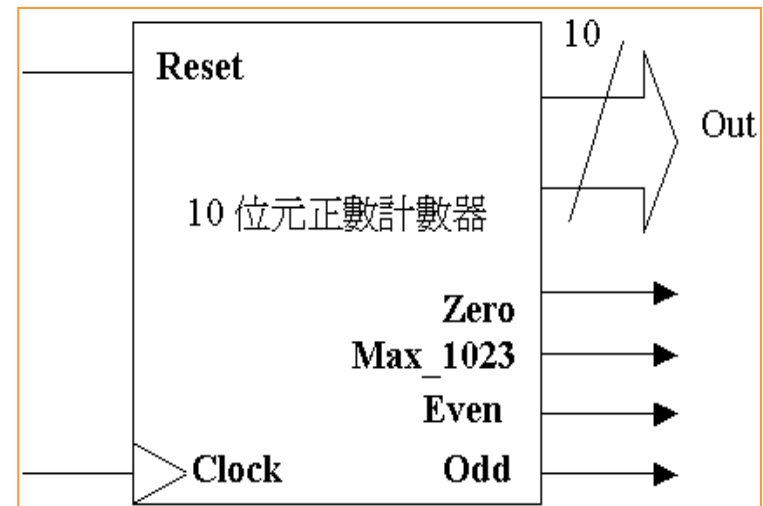
```
input    Clock, Reset;
```

```
output   Zero, Max_1023, Even, Odd;
```

```
wire     Zero, Max_1023, Even, Odd;
```

```
output   [9:0] Out;
```

```
reg      [9:0] Out;
```



Counter (8)

```
assign Zero = !(Out);
assign Max_1023 = & Out;
assign Odd = ^ Out;
assign Even = !(Odd);
```

不寫在always
block內

若計數值為0, 則 Zero = 1
若計數值為1023, 則 Max_1023 = 1
若計數值為偶同位, 則 Even = 1
若計數值為奇同位, 則 Odd = 1

或寫成 $Odd = \sim \wedge Out$

```
always @(posedge Clock or posedge Reset)
```

```
begin
```

```
    if (Reset) // 重置
```

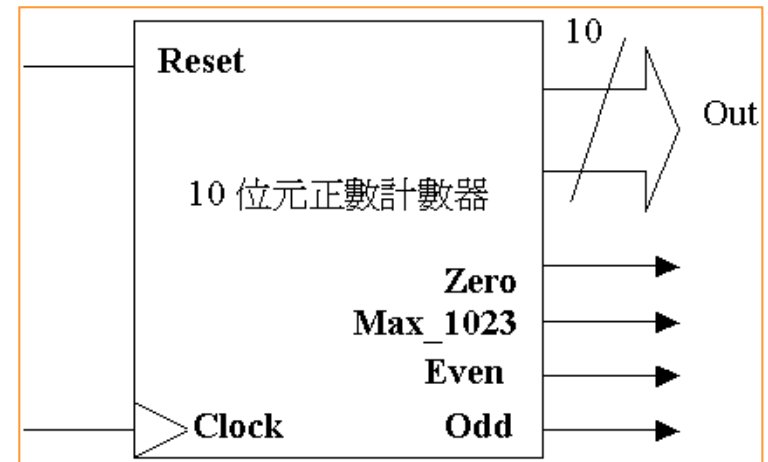
```
        Out = 0; // 清除為 0
```

```
    else
```

```
        Out = Out + 1; // 計數值 加1
```

```
    end
```

```
endmodule
```



Counter (9)

//Counter6.V：上數/下數計數器

//具有「重置」、「載入」、「致能」及「上/下數」訊號控制的

// 8位元(0~255)正緣觸發上數/下數計數器

```
module Counter6 (Clock, Reset, Load, Enable, Up_Down, Data_In, Out);
```

```
input    Clock, Reset, Load, Enable, Up_Down;
```

```
input    [7:0] Data_In;
```

```
output   [7:0] Out;
```

```
reg      [7:0] Out;
```

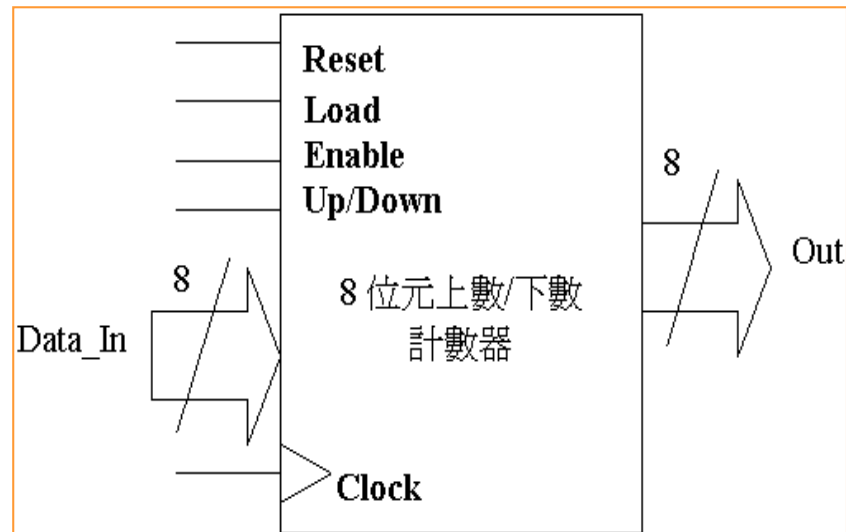
```
always @ (posedge Clock)
```

```
begin
```

```
    if (Reset)
```

```
        Out = 0; // 清除為 0
```

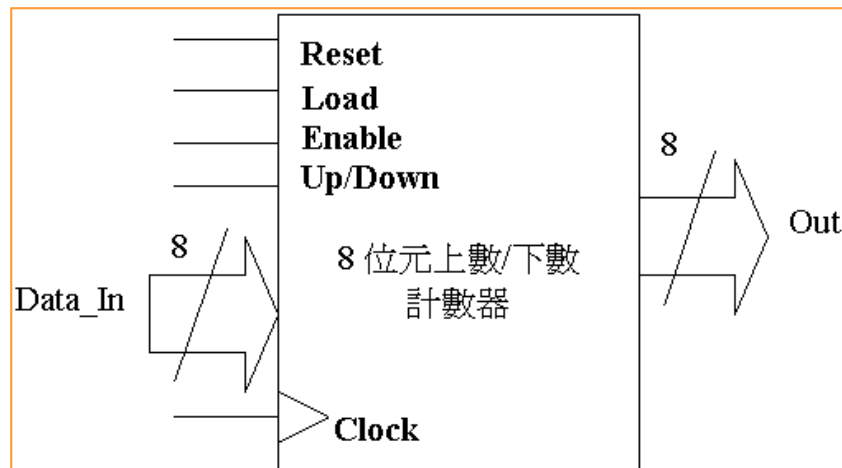
```
    else
```



Priority : Reset > Load > Enable > Up_Down

Counter (10)

```
if (Load) // 載入
    Out = Data_In; // 從輸入埠 Data_In, 載入計數值
else
    if (Enable) // 致能計數
    begin
        if (Up_Down) // 上數/下數
            Out = Out + 1; // 計數值 +1, 上數
        else
            Out = Out - 1; // 計數值 -1, 下數
        end
    end
end
endmodule
```



Frequency Divider (1)

❑ 除頻電路依“除數”大小，可分為下列二種：

- 固定除以某個大小的「除 N 器」(N 為一個正整數值)，
- 可以由使用者自訂 N 的大小的「可程式的除頻器」。

❑ 範例：「除 2 器」，將輸入時脈訊號頻率除以 2 之後再輸出

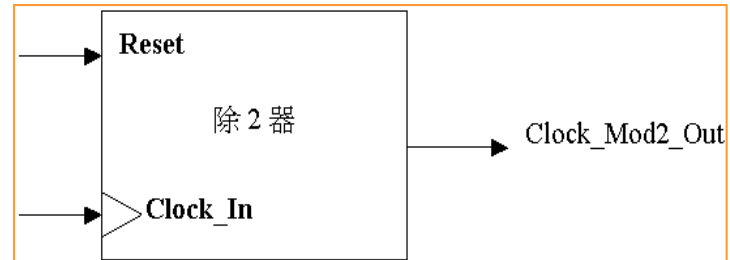
```
module FreqMod2 (Reset, Clock_In, Clock_Mod2_Out);
```

```
input  Reset, Clock_In;
```

```
output Clock_Mod2_Out;
```

```
reg    Clock_Mod2_Out;
```

```
wire   Not_Clock_Mod2_Out;
```



```
    assign Not_Clock_Mod2_Out = !Clock_Mod2_Out;
```

```
always @(posedge Reset or posedge Clock_In)
```

```
begin
```

```
    if (Reset)
```

```
        Clock_Mod2_Out = 0;
```

```
    else
```

```
        Clock_Mod2_Out = Not_Clock_Mod2_Out;
```

```
end
```

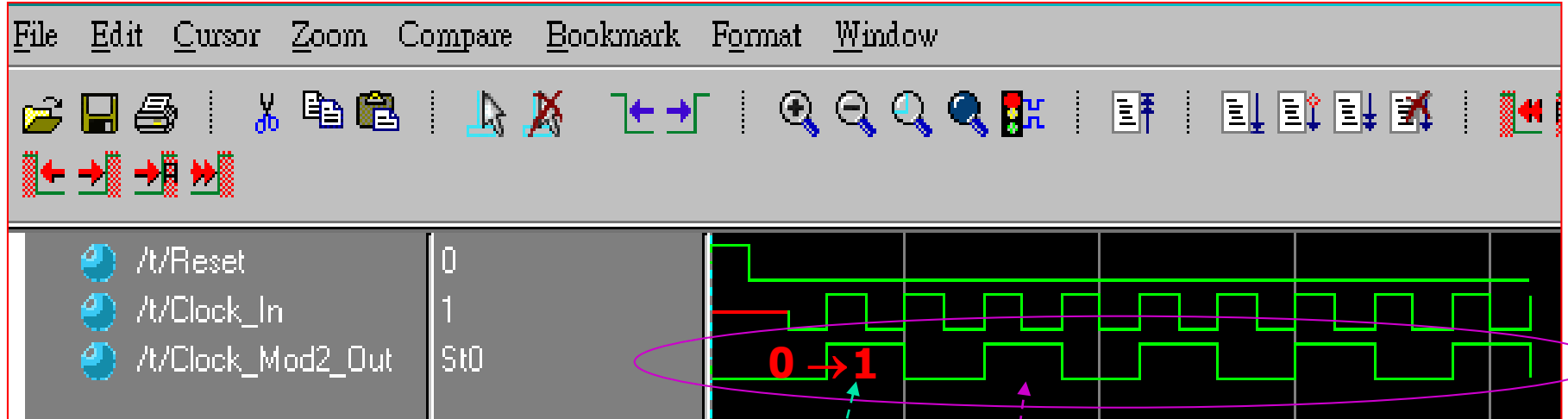
```
endmodule
```

去除assign而令：

`Clock_Mod2_Out = !Clock_Mod2_Out`

Simulation Waveform

4_23_FREQMOD2



```
assign Not_Clock_Mod2_Out = !Clock_Mod2_Out;  
always @(posedge Reset or posedge Clock_In)  
begin  
    if (Reset)  
        Clock_Mod2_Out = 0;  
    else  
        Clock_Mod2_Out = Not_Clock_Mod2_Out;  
end
```

The code is annotated with red and green text and arrows. A red '1' is placed above the '0' in the reset assignment. A green '0(Start)' is placed above the '0' in the reset assignment. A red '1' is placed above the '1' in the toggle assignment. A red '1' is placed above the '1' in the toggle assignment. A red arrow points from the '0(Start)' to the '0' in the reset assignment. A red arrow points from the '1' in the toggle assignment to the '1' in the toggle assignment. A green arrow points from the '0(Start)' to the '1' in the toggle assignment.

Frequency Divider (2)

// 「除 N 器」：可程式的除頻器（將輸入時脈除以N後輸出

```
module FreqModN (LoadModN, Data_N, Clock_In, Clock_ModN_Out);
```

```
input LoadModN;
```

```
input [3:0] Data_N;
```

```
input Clock_In;
```

```
output Clock_ModN_Out;
```

```
reg Clock_ModN_Out;
```

```
reg [3:0] ReadIn_Count, Now_Count;
```

```
reg Is_Count;
```

```
always @(posedge Clock_In) // 正緣脈波訊號
```

```
begin
```

```
if (!LoadModN) // 同步負準位載入
```

```
begin
```

```
    Clock_ModN_Out = 0; // 才剛要開始計數, 所以輸出為0
```

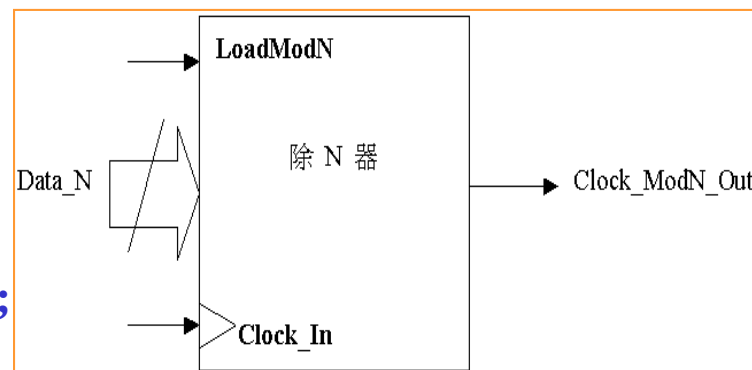
```
    Now_Count = 0; // 目前計數到 0
```

```
    ReadIn_Count = Data_N; // 讀入欲計數的次數 (除頻數)
```

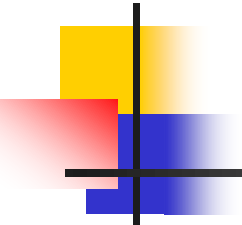
```
    Is_Count = 1; // 設定正在計數旗標為 1
```

```
end
```

```
else
```



初始化

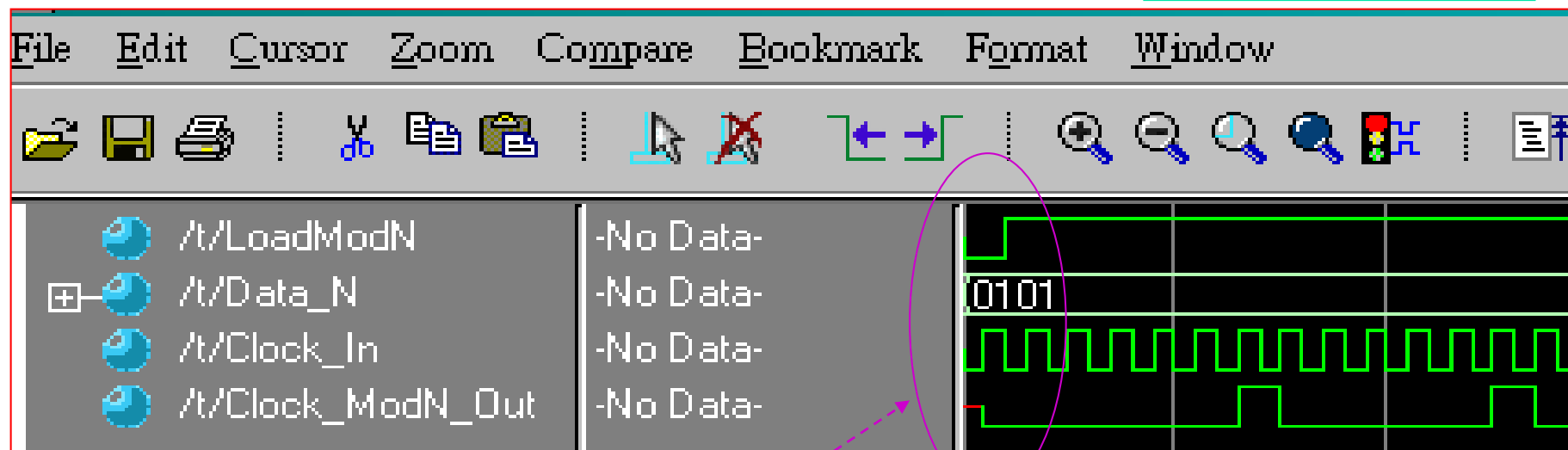


Frequency Divider (3)

```
begin
  if (Is_Count) // 正在計數
  begin
    if (Now_Count == ReadIn_Count) // 如果目前計數的值 = 除頻數
    begin
      Clock_ModN_Out = 1; // 已到除頻值, 所以輸出為1
      Now_Count = 0; // 新設定目前計數值為0
    end
  else
    begin
      Clock_ModN_Out = 0; // 尚未計數到除頻數, 所以輸出為0
      Now_Count = Now_Count + 1; // 目前的計數值 +1
    end
  end
end
end
endmodule
```

Simulation Waveform (1)

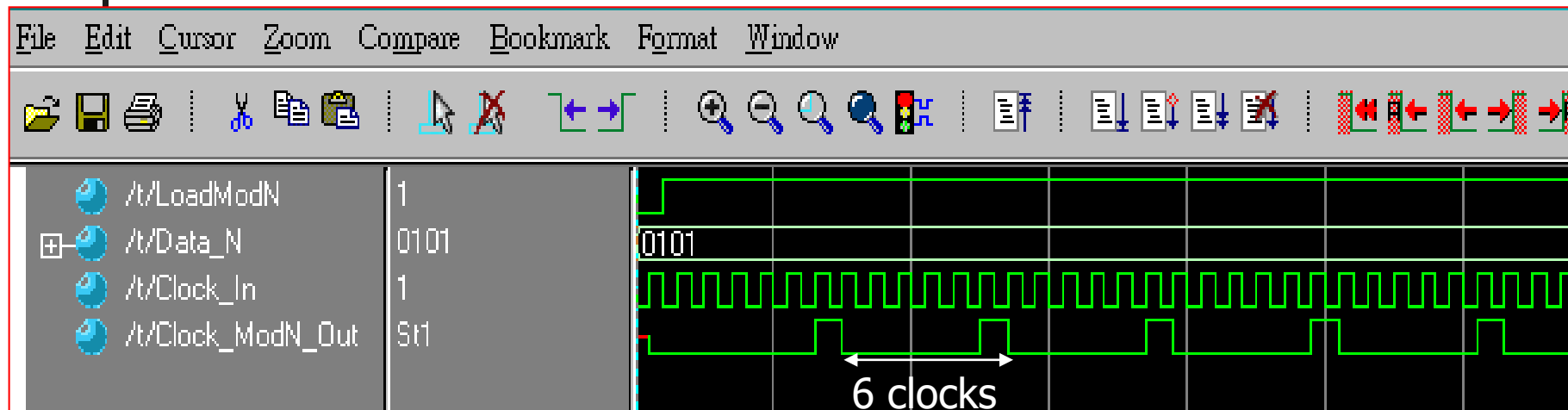
4_24_FREQMODN



```
always @(posedge Clock_In) // 正緣脈波訊號
begin
  if (!LoadModN) // 同步負準位載入
  begin
    Clock_ModN_Out = 0; // 才剛要開始計數, 所以輸出為0
    Now_Count = 0; // 目前計數到 0
    ReadIn_Count = Data_N; // 讀入欲計數的次數 (除頻數=0101=6)
    Is_Count = 1; // 設定正在計數旗標為 1
  end
end
else
```

“÷6” ⇒ “Not ÷5”

Simulation Waveform (2)



```
begin
  if (Is_Count) // 正在計數
  begin
    if (Now_Count == ReadIn_Count - 1) // 如果目前計數的值 = 除頻數
    begin
      Clock_ModN_Out = 1; // 已到除頻值, 所以輸出為1
      Now_Count = 0; // 新設定目前計數值為0
    end
  else
    begin
      Clock_ModN_Out = 0; // 尚未計數到除頻數, 所以輸出為0
      Now_Count = Now_Count + 1; // 目前的計數值 +1
    end
  end
end
end
```

0~5 = 6 clocks

Simulation Waveform (3)

```
module FreqModN (LoadModN, Data_N, Clock_In, Clock_ModN_Out);
```

```
...  
reg      [3:0] ReadIn_Count;
```

```
reg      [2:0] Now_Count;
```

```
reg      Is_Count;
```

Duty cycle = 50%

```
always @(posedge Clock_In) //  $\frac{1}{2}T_{\text{clk}}$   
begin
```

Do it by yourself.

```
...  
end
```

```
else
```

```
begin
```

```
if (Is_Count) //  $\frac{1}{4}T_{\text{clk}}$ ?
```

```
begin
```

```
if (Now_Count == ReadIn_Count[3:1]) //  $\frac{1}{4}T_{\text{clk}}$  =  $\frac{1}{4}T_{\text{clk}}$ 
```

```
begin
```

```
Clock_ModN_Out = !Clock_ModN_Out; //  $\frac{1}{4}T_{\text{clk}}$ ,  $\frac{1}{4}T_{\text{clk}}$ 
```

```
Now_Count = 0; //  $\frac{1}{4}T_{\text{clk}}$ 
```

```
end
```

```
else
```

```
begin
```

```
Clock_ModN_Out = Clock_ModN_Out; //  $\frac{1}{4}T_{\text{clk}}$ ,  $\frac{1}{4}T_{\text{clk}}$ ,  $\frac{1}{4}T_{\text{clk}}$ 
```

```
Now_Count = Now_Count + 1; //  $\frac{1}{4}T_{\text{clk}}$  + 1
```

```
end
```

```
end
```

```
end
```

```
endmodule
```

Simulation Waveform (4)

